Kodna Inteligencija: Temeljni Modeli, Agenti i Primjena

Inženjerska zajednica za softversku inteligenciju

Invalid Date

Table of contents

1 Uvod **3**

2 Temeljni modeli za kod **5**
2.1 Veliki jezični modeli opće namjene . . . 5
2.1.1 Uspon velikih jezičnih modela . . . 5
2.1.2 Arhitekture modela . . . 6
2.1.3 Multimodalnost . . . 8
2.1.4 Ograničenja velikih jezičnih modela opće namjene . . . 9
2.2 Veliki jezični modeli za kod . . . 10
2.2.1 Zatvoreni modeli za kod . . . 10
2.2.2 Otvoreni modeli za kod . . . 12

3 Zadaci, mjerila i evaluacija koda **13**
3.1 Metrike evaluacije . . . 14
3.1.1 Pass@k . . . 14
3.1.2 Točnost (Accuracy) . . . 14
3.1.3 BLEU i ROUGE . . . 14
3.1.4 Tree-based Metrics . . . 15
3.1.5 Execution-based Metrics . . . 15
3.1.6 Embedding-based Metrics . . . 15
3.1.7 Human Evaluation . . . 16
3.2 Zadaci na razini funkcije . . . 16

4 Usklađivanje **16**
4.1 Nadzirano fino ugađanje (SFT) . . . 17
4.1.1 Jednokružno nadzirano fino ugađanje . . . 17
4.1.2 Višekružno nadzirano fino ugađanje . . . 18

4.1.3 SFT za zadatke na razini repozitorija . . . 18
4.1.4 Metode bazirane na rezoniranju . . . 18
4.1.5 Strategije treniranja . . . 19
4.1.6 Izazovi u SFT-u . . . 19
4.2 Hladni start i destilacija podataka rezoniranja . . . 20
4.3 Višejezično razumijevanje i generiranje koda . . . 20
4.4 Multimodalno razumijevanje i generiranje koda . . . 21
4.4.1 Generiranje korisničkog sučelja . . . 21
4.4.2 Web utjelovljena inteligencija . . . 22
4.4.3 Generiranje artefakata softverskog inženjerstva . . . 22
4.5 Pojačano učenje u kodnoj inteligenciji . . . 22
4.5.1 Algoritmi pojačanog učenja . . . 22
4.5.2 RL za generiranje koda . . . 23
4.5.3 RL za razumijevanje koda . . . 23
4.5.4 RL za softversko inženjerstvo . . . 24
4.5.5 RL za sigurnost koda . . . 24
4.6 Pojačano učenje s provjerljivim nagradama . . . 24
4.6.1 Skupovi podataka prikladni za RLVR u kodnim zadacima . . . 25

5 Agenti za softversko inženjerstvo 26
5.1 Agenti kroz životni ciklus softverskog inženjerstva . . . 26
5.1.1 Inženjerstvo zahtjeva . . . 26
5.1.2 Razvoj softvera . . . 27
5.1.3 Testiranje softvera . . . 27
5.1.4 Održavanje softvera . . . 27
5.2 Opći kodovi agenti u softverskom inženjerstvu . . . 28
5.3 Tehnike treniranja za SWE agente . . . 28
5.4 Budući trendovi SWE agenata . . . 28

6 Kod za generalističke agente 29
6.1 Kod kao protokoli interakcije . . . 30
6.1.1 Korištenje alata . . . 30
6.1.2 Protokol konteksta modela (MCP) . . . 31
6.1.3 Koordinacija više agenata . . . 31
6.2 Kod kao agentske sposobnosti . . . 31
6.2.1 Razmišljanje u kodu . . . 31
6.2.2 Djelovanje u kodu . . . 32
6.2.3 Pamćenje kodom . . . 33
6.3 Kod kao sučelja okruženja . . . 33
6.3.1 Kod kao simulacijski poligon . . . 33
6.3.2 Agenti za korištenje računala . . . 34

7 Sigurnost kodnih LLM-ova **35**
7.1 Sigurnost pretreniranja kodnih LLM-ova . . . 36
7.1.1 Porijeklo podataka, sigurnost i usklađenost s licencama . . . 36
7.1.2 Revizija i čišćenje podataka za treniranje . . . 36
7.1.3 Robusnost protiv neprijateljskih transformacija koda . . . 36
7.2 Sigurnost posttreniranja kodnih LLM-ova . . . 37
7.2.1 Podatci kao temelj za sigurnost . . . 37
7.2.2 Sigurnosno nadzirano fino ugađanje . . . 37
7.2.3 Sigurnosno usklađivanje kodiranja putem pojačanog učenja . . . 37
7.3 Tehnike crvenog tima za kodne LLM-ove . . . 38
7.3.1 Manipulacija na razini upita . . . 38
7.3.2 Semantička i kontekstualna manipulacija . . . 38
7.3.3 Manipulacija agentskog tijeka rada . . . 38
7.4 Strategije ublažavanja za rizike kodiranja . . . 38
7.4.1 Temelji u sigurnim okruženjima za izvršavanje . . . 39
7.4.2 Proaktivna obrana i validacija prije izvršavanja . . . 39
7.4.3 Nadzor u izvršavanju i utemeljenje namjere . . . 39

8 Recepti za treniranje **39**
8.1 Distribuirani okviri za treniranje . . . 40
8.2 Smjernice za pretreniranje . . . 40
8.3 Smjernice za fino ugađanje . . . 41
8.4 Smjernice za pojačano učenje . . . 42

9 Veliki jezični modeli za kod u primjeni **42**
9.1 Razvojni pomoćnici integrirani u IDE . . . 42
9.2 Platforme za kodiranje u oblaku . . . 44
9.3 Autonomni agenti bazirani na terminalu . . . 44
9.4 Aplikacije za popravak i verifikaciju koda . . . 45
9.5 Pregled zahtjeva za spajanje i osiguranje kvalitete . . . 45

Reference **46**

1 Uvod

Dvadeset devetog listopada 1969. godine, u laboratoriju UCLA-a u Los Angelesu, mladi istraživač po imenu Charley Kline sjeo je za računalni terminal i pokušao poslati jednu jedinu riječ drugom računalu udaljenom oko četiri stotine kilometara, na Stanfordu. Riječ je bila *login*. Sustav se srušio nakon dva slova. Ekran je pokazao samo *lo*. Ipak, ta dva slova putovala su kroz telefonske linije poput prvih koraka djeteta koje još ne zna da će jednog dana pretrčati maraton. Pet i pol desetljeća kasnije, strojevi ne samo da šalju poruke, nego pišu softver, ispravljaju greške u tuđem kodu i sudjeluju u arhitektonskim odlukama koje oblikuju digitalne

proizvode kojima se koriste milijarde ljudi. Put od ta dva slova do današnjeg stanja stvari nije bio linearan, ali je bio neizbježan.

Zamislite tvornicu u kojoj radnici ne sastavljaju dijelove rukama, nego riječima. Jedan radnik kaže *napravi dugme koje mijenja boju kada ga korisnik pritisne*, a stroj na drugoj strani tvornice proizvede gotov komad softvera koji radi točno to. Upravo se to događa danas kada veliki jezični modeli (engl. *Large Language Models* ili LLM) prevode ljudski jezik u izvršivi kod. Ova analogija s tvornicom nije pretjerana. Alati poput GitHub Copilota, Cursora, Claude Codea i CodeWhisperera funkcioniraju upravo tako, pretvarajući opis zadatka napisan običnim riječima u funkcionalne programe koje računalo može izvršiti. Transformacija je toliko duboka da mijenja sam pojam programiranja, jer više nije potrebno poznavati svaku sintaktičku nijansu programskog jezika da biste stvorili nešto korisno.

Zašto bi vas to trebalo zanimati? Zato što se evolucija razvoja softvera može podijeliti u jasne epohe, a mi upravo živimo na prijelazu iz jedne u drugu. Prva epoha trajala je od šezdesetih do osamdesetih godina dvadesetog stoljeća i u njoj su programeri ručno pisali svaki redak koda. Druga epoha donijela je integrirane razvojne alate poput IDE-eva i sustava za upravljanje verzijama koji su učinili ručni rad produktivnijim. Treća epoha, od devedesetih do dvadesetih godina ovog stoljeća, obilježena je razvojnim okvirima (engl. *frameworks*) koji su apstrahirali rutinske zadatke. No četvrta epoha, ona u kojoj se nalazimo od otprilike 2020. godine, radikalno je drugačija. U njoj umjetna inteligencija ne samo da pomaže programerima, nego počinje samostalno obavljati značajne dijelove razvojnog procesa. I upravo je to epoha o kojoj govori ovaj tekst.

Krajolik suvremenih modela za kodiranje otkriva stratešku podjelu između dvaju pristupa. S jedne strane nalaze se modeli opće namjene poput GPT serije, Claudea i LLaMA obitelji, koji nude širinu jer su trenirani na golemim količinama prirodnog jezika zajedno s programskim kodom. S druge strane stoje specijalizirani modeli za kod poput StarCodera, Code LLaMA-e, DeepSeek-Codera i QwenCodera, koji postižu vrhunske rezultate na mjerilima specifičnima za programiranje zahvaljujući ciljanom pretreniranju na programerskim podatcima. Poboljšanje performansi, od jednoznamenkastih postotaka do više od 95% uspješnosti na standardiziranim testovima poput HumanEvala, odražava i algoritmske inovacije i dublja uvida u strukturu samog koda Chen i ostali (2021).

Usprkos intenzivnom istraživačkom naporu i brzoj komercijalnoj primjeni, u literaturi postoji praznina između širine inovacija i dubine sustavne analize. Postojeći pregledi uglavnom su pristupali panoramski, pokrivajući široke kategorije zadataka ili se fokusirajući na starije generacije modela, dok suvremeni napredci ostaju nedovoljno sintetizirani. Posebno su nedovoljno istražene sofisticirane strategije kuriranja podataka, metode usklađivanja ponašanja modela s namjerama programera, napredne paradigme poticanja (engl. *prompting*) uključujući lančano rezoniranje (engl. *chain-of-thought*) Wei i ostali (2022), te pojava autonomnih agenata za kodiranje sposobnih za višekorakno rješavanje problema. Ovaj tekst nastoji upravo tu prazninu popuniti, nudeći sustavni pregled cjelokupnog životnog ciklusa modela za kodiranje, od početnog prikupljanja i čišćenja podataka, preko pretreniranja i arhitektonskih inovacija, do naprednih metoda finoštimanja i primjene u stvarnom softverskom inženjerstvu. Svrha nije pretvoriti vas

u stručnjake za strojno učenje, ali jest omogućiti vam da razumijete koncept dovoljno duboko da u profesionalnom razgovoru možete postaviti pravo pitanje u pravom trenutku.

2 Temeljni modeli za kod

Prije nego što zaronimo u specijalizirane modele za programski kod, nužno je razumjeti tlo na kojem su izgrađeni. Veliki jezični modeli opće namjene su temelj, osnova na kojoj počiva sve što slijedi. Zamislite da gradite kuću. Temelj sam po sebi ne izgleda posebno, to su beton i armaturu zakopani u zemlju, i nitko ga ne fotografira za naslovnicu arhitektonskog časopisa. Ali bez njega, ni najljepša fasada ne može stajati. Veliki jezični modeli opće namjene upravo su taj temelj za sve specijalizirane sustave koji generiraju, analiziraju i popravljaju kod.

2.1 Veliki jezični modeli opće namjene

2.1.1 Uspon velikih jezičnih modela

Prije nego što su transformeri stupili na scenu, napredak u umjetnoj inteligenciji bio je rascjepkan. Postojali su specijalizirani sustavi za strojno prevođenje, zasebni cjevovodi za dijalog, posebni motori za sintezu programa. Svaki je radio pristojno u svojoj niši, ali nijedan nije mogao izaći iz nje. Zamislite grad u kojem svaka četvrt ima vlastitu željezničku prugu, ali nijedna pruga nije spojena s drugom. Transformerska arhitektura, predstavljena 2017. godine u čuvenom radu *Attention Is All You Need* Vaswani i ostali (2017), bila je upravo ta spojna pruga. Umjesto da gradi zasebne sustave za svaki zadatak, transformersko pretreniranje i prijenos znanja ujedinili su te raznorodne niti u jedinstven, skalabilan okvir koji se mogao prilagoditi različitim zadacima i modalitetima.

Zakoni skaliranja pokazali su nešto važno. Sa svakim povećanjem broja parametara modela, količine podataka i računalne snage, performanse su rasle na predvidljiv način. Ali najznačajnije su takozvane emergentne sposobnosti, vještine koje se pojavljuju tek na određenim skalama, kao da model na nekom pragu veličine naglo nauči nešto što manje verzije naprosto ne mogu. Doduše, recentna istraživanja ukazuju da bi dio tih emergentnih sposobnosti mogao biti artefakt načina mjerenja, što daje nijansiraniju sliku o dobrobitima puke veličine. No činjenica ostaje. Modeli su, rastom, stekli dvije posebno značajne klase sposobnosti koje nikad nisu bile eksplicitno programirane.

Prva je sposobnost kodiranja. Modeli opće namjene pokazali su iznenađujuću vještinu u generiranju programskog koda, što je kataliziralo razvoj modela eksplicitno treniranih na kodu. OpenAI-jev Codex demonstrirao je funkcionalno generiranje koda iz opisa na prirodnom jeziku i uveo standardizirane načine evaluacije poput HumanEvala. Paralelno, DeepMindov AlphaCode pokazao je da kombinacija masovnog uzorkovanja i filtriranja može dostići razinu natjecateljskog programiranja približnu medijalnom ljudskom natjecatelju u simuliranim Codeforces uvjetima Y. Li i ostali (2022). Ovi rezultati utvrdili su da jezično modeliranje i sinteza koda dijele

strukturu koju je moguće iskoristiti, čineći LLM-ove odmah upotrebljivima za zadatke od generiranja predložaka koda do rješavanja algoritmskih problema.

Druga emergentna sposobnost je važnija. Kada su LLM-ovi opremljeni vanjskim alatima, memorijom i zatvorenom petljom rezoniranja, počeli su nalikovati na donositelje odluka, a ne na statičke prediktore. Metode poput ReActa Yao i ostali (2023) isprepliću tragove rezoniranja s akcijama u okruženju kako bi planirali, prikupljali informacije i ispravljali smjer. Komplementarni pristupi poput Toolformera Schick i ostali (2024) pokazali su da modeli mogu naučiti kada i kako pozivati vanjske programska sučelja (engl. *API*) na samoupravljan način, poboljšavajući pouzdanost na zadacima koji profitiraju od kalkulatora, pretraživanja ili dohvaćanja podataka. Agenti za softversko inženjerstvo (engl. *SWE agents*) postigli su zapažen napredak.

Zajedno gledano, ovi razvoji označavaju čist prekid s uskim, zadatku specifičnim sustavima i pomak prema općem sustavu za kodiranje koji pruža jedinstvenu podlogu za jezik, programiranje i rezoniranje posredovano alatima. No upravo ta širina otkriva i granice u točnosti, sigurnosti i pouzdanosti na razini cijelih sustava u profesionalnim softverskim okruženjima. I upravo te granice motiviraju specijalizirane modele za kodiranje i agente kojima se bavi ostatak ovog teksta.

2.1.2 Arhitekture modela

Uz rast u razmjeru i količini podataka, inovacije u arhitekturi modela bile su jedan od ključnih stupova brzog napretka velikih jezičnih modela. Zamislite tvorničku halu u kojoj rade stotine radnika. U početku, svaki radnik sudjeluje u svakom koraku proizvodnje, što je pouzdano ali skupo. To je način na koji funkcioniraju gusti modeli (engl. *dense models*). Zatim netko dolazi na ideju da se radnici specijaliziraju, svaki za određeni tip zadatka, i da se na pokretnu traku pozivaju samo oni koji su za taj konkretni korak najpotrebniji. To je načelo mješavine stručnjaka (engl. *Mixture of Experts* ili MoE). A onda se pojavljuju i potpuno drugačije ideje organizacije, od rekurentnih linija do difuzijskih postupaka koji materijal oblikuju postupnim dotjerivanjem umjesto sekvencijalnim slaganjem. Evolucija arhitektura upravo je priča o tim različitim načinima organizacije rada.

Gusti modeli. Transformerski model ostaje temelj modernih LLM-ova, koristeći guste arhitekture u kojima svaki parametar sudjeluje u obradi svakog tokena. Ovaj dizajn, izgrađen na slojevima mehanizma pažnje i unaprijednih mreža, omogućio je napredak u hvatanju dugoročnih ovisnosti i u zadacima obrade prirodnog jezika. Modeli poput LLaMA obitelji i njezinih nasljednika pokazali su da otvoreni modeli visoke kvalitete mogu konkurirati vlasničkim sustavima, skalirajući se od 7 milijardi do 70 milijardi parametara. Serija GLM proširila je guste arhitekture u dvojezične i višejezične domene, dok je obitelj Qwen naglasila snažne performanse u razumijevanju i generiranju sa skalabilnim gustim modelima. Mistral je istaknuo kako pažljivo inženjerstvo, poput grupirane pažnje za upite (engl. *grouped query attention*

ili GQA), može postići konkurentne rezultate s manje parametara. Gusti modeli su računski zahtjevni, ali nastavljaju evoluirati prema većoj učinkovitosti i svestranosti.

Mješavina stručnjaka (MoE). MoE proširuje kapacitet modela kroz uvjetno računanje bez proporcionalnog povećanja aktivirane računalne snage. Svaki token usmjerava se na mali broj stručnjaka, obično najsposobnijih k stručnjaka, za izračun proslijeđivanja, zamjenjujući rijetku aktivaciju za veći efektivni kapacitet. U zajednici otvorenog koda, serija Mixtral učinila je usmjeravanje na dva stručnjaka de facto inženjerskim standardom. Varijanta 8×7B demonstrirala je da aktiviranje manje parametara može nadmašiti veće guste modele, a naknadna 8×22B dodatno je pomaknula granice sposobnosti i propusnosti. Serija Qwen uvela je MoE varijante kroz svoje verzije 1.5, 2.5 i 3. DeepSeek je sistematizirao učinkovit dizajn rijetkih stručnjaka i višeglave latentne pažnje (engl. *Multi-head Latent Attention* ili MLA) u svojoj V2/V3 seriji, gdje V2 ima 236 milijardi ukupnih parametara s oko 21 milijardom aktiviranih, a V3 ima 671 milijardu ukupnih parametara s oko 37 milijardi aktiviranih DeepSeek-AI (2024). DeepSeek R1 nadalje je nadogradio V3-Base pojačanim učenjem za poboljšanja lančanog rezoniranja DeepSeek-AI (2025). GLM-4.5 primijenio je MoE velikog razmjera, integriravši hibridne načine rezoniranja u jedinstven model za kodiranje, rezoniranje i agentske primjene. Cijela LLaMA-4 serija također usvaja MoE arhitekturu. Ukratko, MoE je postao jedna od glavnih arhitektura za optimizaciju omjera efektivnog kapaciteta i u praksi sinergijski djeluje s rukovanjem dugim kontekstima, kompresijom KV predmemorije i predikcijom više tokena, tvoreći učinkovitu paradigmu za velika produkcijska okruženja.

Rekurentni modeli. Arhitekture rekurentnog tipa ponovno posjećuju sekvencijalno modeliranje kako bi smanjile memoriju i latenciju zadržavajući paralelno treniranje. RWKV spaja paralelizabilno treniranje nalik transformerima s rekurentnim zaključivanjem, aktivirajući stanje konstantne veličine u svakom koraku tako da dekodiranje skalira linearno i može se približiti kvaliteti transformera pri sličnim veličinama. Retentivne mreže (engl. *RetNet*) zamjenjuju pažnju operatorom zadržavanja koji podržava potpuno paralelno treniranje i rekurentno ili blokovno rekurentno zaključivanje, dajući obradbu dugih sekvenci u linearnom vremenu sa snažnim rezultatima jezičnog modeliranja. Mamba Gu i Dao (2024) uvodi selektivne modele prostora stanja čiji parametri ovise o ulazu, omogućujući dekodiranje u linearnom vremenu i konkurentne performanse na jezičnim zadacima uz visoku propusnost. Srodni operatori dugog dosega poput Hyene koriste implicitno parametrizirane duge konvolucije s kontrolnim mehanizmima kako bi postigli kvalitetu pažnje uz subkvadratni trošak, što omogućuje duljine konteksta daleko iznad standardnih režima pažnje. Dodatno, DeltaNet uvodi hardverski učinkovit način za paralelizaciju linearnih transformera, dok Gated DeltaNet kombinira kontrolne mehanizme s delta ažuriranjem za bolje upravljanje memorijom, dosljedno nadmašujući Mambu-2 i DeltaNet na dugokontekstnim zadacima.

Modeli bazirani na difuziji. Difuzijski jezični modeli zamjenjuju dekodiranje slijeva nadesno iterativnim koracima odnošumljivanja koji pročišćavaju bučnu sekvencu u tečan tekst, omogućujući globalnu kontrolu nad atributima i strukturom. Temeljni radovi na diskretnoj difuziji formalizirali su procese korupcije i odnošumljivanja izravno u prostoru tokena. Nedavno su se pojavili veći napori koji guraju difuzijske jezične modele izvan malih prototipova. LLaDA

trenira difuzijske modele za jezik od nule putem rasporeda maskiranja i obrnutog odnošumljivanja s običnim transformerom, postižući konkurentnost s autoregresivnim baznim modelima slične veličine. Na komercijalnoj strani, Mercury Coder oblikuje kodiranje kao paralelno višetokensko odnošumljivanje i reklamira znatne prednosti u brzini u odnosu na autoregresivne modele. Gemini Diffusion istražuje difuziju za generiranje teksta, signalizirajući kontinuirani interes za neautoregresivno dekodiranje na produkcijskoj skali. Iako difuzijski jezični modeli nude mogućnost kontrole i paralelizabilne ciljeve treniranja, obično zahtijevaju mnogo koraka uzorkovanja, što motivira istraživanje bržih uzorkivača i hibridnih autoregresivno-difuzijskih dekodera.

Hibridne arhitekture. Hibridne arhitekture isprepliću komplementarne operatore sekvenci, obično kombinirajući transformersku pažnju s blokovima prostora stanja ili rekurentnim blokovima, često uz MoE unaprijedne mreže, kako bi u jednom skupu uskladili kvalitetu, duljinu konteksta i propusnost. Jamba je kanonički primjer koji isprepliće slojeve transformera i Mambe s MoE, postižući visoku propusnost na dugim kontekstima zadržavajući snažne performanse. U Qwen liniji, Qwen3-Next usvaja hibridni dizajn pažnje koji miješa linearne operatore u stilu Gated DeltaNeta s kontroliranom pažnjom i rijetkom aktivacijom MoE. DeepSeek V3 uvodi MLA s DeepSeek-MoE za učinkovito treniranje i zaključivanje. Hibridne arhitekture postale su standard za moderan razvoj velikih modela.

Arhitektura modela diversificirala se od univerzalnog gustog transformera prema alatnici koja uključuje rijetke stručnjake, rekurentnost i prostor stanja, difuziju i učinkovite mehanizme pažnje. Ovi izbori omogućuju praktičarima da balansiraju kapacitet, latenciju i duljinu konteksta, pružajući sposobnosti koje podupiru i opće LLM-ove i specijalizirane sustave za kodiranje.

> **Note**
>
> Za vašu profesionalnu praksu: Kada na sastanku netko kaže da prebacuju infrastrukturu na MoE arhitekturu ili da razmatraju Mambu za zaključivanje u stvarnom vremenu, razumjet ćete da se radi o temeljno različitim načinima organiziranja računanja unutar modela, svaki s vlastitim prednostima u trošku, brzini i kvaliteti.

2.1.3 Multimodalnost

Do sada smo govorili o modelima koji rade s tekstom, bilo da je to prirodni jezik ili programski kod. Ali stvarni svijet razvoja softvera nije čisto tekstualan. Zamislite da radite na mobilnoj aplikaciji i dizajner vam pošalje skicu korisničkog sučelja, dijagram arhitekture sustava ili snimku zaslona postojeće stranice s napomenom *napravi nešto ovako*. U tom trenutku, model koji razumije samo tekst nalikuje kuharu koji može čitati recepte, ali je slijep. Može raditi s riječima, ali ne vidi što je na tanjuru.

Upravo tu nastupa multimodalnost. Modeli za kodiranje trebaju obrađivati vizualne informacije poput dijagrama, snimaka zaslona i elemenata korisničkog sučelja da bi razumjeli i generirali kod u stvarnim scenarijima. Ove sposobnosti čine temelj za radne tijekove orijentirane na kod. Najnovije generacije modela, poput Gemini serije i Claudea, integrirale su vizualno razumijevanje izravno u svoju arhitekturu, što znači da mogu primiti sliku korisničkog sučelja i generirati HTML ili CSS koji ga reproducira, ili analizirati dijagram toka podataka i predložiti odgovarajuću arhitekturu koda.

Zašto bi vas multimodalnost trebala zanimati? Zamislite budući radni dan u kojem projekt menadžer dijeli dijagram u Figmi i pita može li AI izravno iz tog dijagrama generirati početnu strukturu koda. Ili zamislite da testirate aplikaciju, napravite snimku zaslona greške i tražite od modela da dijagnosticira problem na temelju onoga što vidi. To više nije znanstvena fantastika. To je smjer u kojem se ova tehnologija kreće, i razumijevanje multimodalnosti ključno je za razumijevanje zašto sljedeća generacija alata za kodiranje izgleda upravo onako kako izgleda.

2.1.4 Ograničenja velikih jezičnih modela opće namjene

Sve što smo dosad opisali moglo bi stvoriti dojam da su veliki jezični modeli opće namjene rješenje za svaki problem u razvoju softvera. To bi bio opasan zaključak. Zamislite liječnika opće prakse. On zna dovoljno o svakom organu da prepozna simptome, ali kada trebate operaciju srca, tražite kardiokirurga. Veliki jezični modeli opće namjene su upravo taj liječnik opće prakse. Njihova širina je impresivna, ali dubina, robusnost i usklađenost s domenom često nisu dovoljne za profesionalno softversko inženjerstvo. Pogledajmo konkretno gdje se javljaju pukotine.

Specijalizacija i točnost. Usprkos svojoj širini, modeli opće namjene često nemaju dubinu potrebnu za profesionalno softversko inženjerstvo. Mogu proizvesti kod koji povrinski izgleda ispravno, ali ne zadovoljava domenske ograničice poput suptilnih ugovora programskih sučelja (engl. *API contracts*), sigurnosnih politika, i teško održavaju invarijante kroz velike sustave. Dokazi iz evaluacija na razini čitavih repozitorija dodatno ukazuju da rješavanje problema iz stvarnog svijeta ostaje izazovno čak i za snažne modele i agentske alatne lance.

Sigurnost i pouzdanost. Rastući korpus empirijskih studija pokazuje da funkcionalno ispravan kod iz općih LLM-ova može i dalje biti nesiguran. Evaluacije velikog razmjera koje uključuju više od stotinu modela kroz osamdeset zadataka izvješćuju da otprilike 45% generacija sadrži poznate ranjivosti, s malo poboljšanja kod novijih ili većih modela. Razmislite o tome. Gotovo polovica koda koji ovi modeli proizvode može sadržavati sigurnosne propuste. To je kao da vam građevinski inženjer kaže da svaki drugi most koji dizajnira ima strukturalni problem. Ne bi vas utješila činjenica da most izgleda lijepo.

Razumijevanje na razini repozitorija. Čak i s proširenim kontekstnim prozorima, opći LLM-ovi ne koriste robustno vrlo duge ulaze. Performanse se pogoršavaju kada se relevantna informacija nalazi u sredini konteksta umjesto blizu njegovih krajeva, a mjerila na razini repozitorija koja pokrivaju zadatke poput višedatotečnog dovršavanja, dohvata i uređivanja

otkrivaju ustrajne poteškoće u praćenju međudatotečnih ovisnosti i globalnom rezoniranju. Zamislite da čitate roman od tisuću stranica, ali zaboravljate sve što je pisalo na sredini knjige. Upravo to se događa velikim jezičnim modelima kada obrađuju velike baze koda.

Multimodalno trenje. Opći multimodalni modeli pružaju korisnu percepciju za snimke zaslona, dokumente i dijagrame, ali finogranulirana hijerarhija korisničkog sučelja i semantika interakcije ostaju slabe točke. Nedavne analize razumijevanja grafičkih korisničkih sučelja (engl. *GUI*) bilježe da postojeći sustavi često specijaliziraju uske podzadatke umjesto da postižu holistično i dosljedno razumijevanje zaslona, što ograničuje stabilne prijelaze od percepcije do akcije u stvarnim primjenama.

Agentska ograničenja. Za postavke proširene alatima, evaluirani agenti i dalje zakazuju zbog krhkog dugoročnog rezoniranja, donošenja odluka i praćenja uputa. Sustavne evaluacije ističu značajne praznine kroz interaktivna okruženja i domene, a nove dijagnostike dokumentiraju halucinacije s alatima, poput pogrešnog odabira alata, netočnog vremena pozivanja ili fabriciranih rezultata alata. Ove studije nadalje predlažu usklađivanje pouzdanosti kako bi se ublažili takvi problemi.

Ukupno gledano, širina bez usklađenosti s domenom vodi do praznina u dubini, pouzdanosti i koherentnosti na razini sustava. Rješavanje ovih ograničenja motivira specijalizirano pretreniranje za kodiranje, kuriranje podataka, sigurnosno usklađivanje i evaluaciju, s modelima optimiziranima da djeluju kao stručni programeri.

2.2 Veliki jezični modeli za kod

U prethodnom potpoglavlju upoznali smo velike jezične modele opće namjene, one koji znaju mnogo o svemu, ali nisu specijalisti ni za što. Sada prelazimo na njihove specijalizirane srodnike, modele koji su od samog početka dizajnirani, trenirani i optimizirani za pisanje, razumijevanje i popravljanje programskog koda. Razlika između ta dva svijeta najlakše se razumije sljedećom analogijom. Zamislite dva kuhara. Prvi je talentirani amater koji kuha odlična jela iz različitih kuhinja i uvijek napravi nešto ugodno za nepce. Drugi je profesionalni slastičar koji je proveo deset godina usavršavajući isključivo croissante. Obojica su vješti u kuhinji, ali kada trebate savršeni croissant za pariški bistro, znate koga ćete pozvati. Specijalizirani modeli za kod upravo su ti slastičari. Oni nisu nužno bolji u pisanju ljubavnih pisama ili objašnjavanju kvantne fizike, ali kada im date zadatak da napišu funkciju u Pythonu, poprave grešku u JavaScriptu ili refaktoriraju cijeli repozitorij, njihova izvedba nadmašuje generaliste jer je svaki korak njihovog treniranja bio usmjeren prema tom cilju.

2.2.1 Zatvoreni modeli za kod

Zatvoreni modeli za kod evoluirali su od osnovnog generiranja koda do agentskih sustava sa sposobnostima na razini cijelih repozitorija. Njihova povijest počinje u laboratorijima nekolicine

tvrtki koje su imale resurse za treniranje golemih modela, a završava u integriranim razvojnim okruženjima na kojima hoje rade milijuni programera diljem svijeta.

Model Obitelj	Ključne Inovacije	Trenutni Status
GPT serija (OpenAI)	Pretreniranje s malo uzoraka, Codex, multimodalnost, o-serija s promišljanjem	GPT-5 s vrhunskim rezultatima
PaLM/Gemini (Google)	Nativna multimodalnost, dugi konteksti, rijetki stručnjaci	Gemini 2.5 za kodne primjene
Claude (Anthropic)	Ustavna AI, dugi konteksti, agentske sposobnosti, korištenje alata	Claude 4/4.5 s alatnim lancima
Grok serija (xAI)	Korištenje alata, načini promišljanja, specijalizirane kodne krajnje točke	Grok-4 sa kodnom specijalizacijom
AlphaCode (DeepMind)	Masovno stohastičko uzorkovanje, filtriranje bazirano na izvršavanju	AlphaCode 2 s Gemini politikama

OpenAI-jeva GPT serija snažno je oblikovala inteligenciju koda Chen i ostali (2021). Rani GPT-1 i GPT-2, koji su bili otvoreni, potvrdili su valjanost generativnog pretreniranja. Vlasnički nasljednici, GPT-3, Codex, GPT-4 i o-serija usmjerena na rezoniranje, proširili su raspon od učenja iz konteksta i sinteze koda do multimodalnog korištenja i popravaka na razini repozitorija. GPT-3 je demonstrirao da se modeli mogu prilagoditi iz samo nekoliko primjera bez ažuriranja parametara. Codex je nastavio treniranje GPT-3 na golemim GitHub bazama koda i pogonio je GitHub Copilot, čime je uspostavljen prvi masovni most između jezičnog modela i stvarnog programerskog alata. InstructGPT uveo je usklađivanje s ljudskim povratnim informacijama Ouyang i ostali (2022). ChatGPT je to nadogradio stabilnim višekružnim dijalogom i sigurnosnim mehanizmima. GPT-4 je unaprijedio rezoniranje i sintezu koda, dok su varijante poput GPT-4 Turbo, GPT-4o i GPT-4o mini optimizirale učinkovitost, multimodalnost i troškovnu pristupačnost. O-serija (o1, o3) uvela je korak po korak unutarnje promišljanje, posebno za složene zadatke kodiranja. GPT-5 i GPT-5-Codex predstavljaju najnoviju generaciju koja spaja vrhunske rezultate na mjerilima s interaktivnim, povratnom informacijom vođenim radnim procesima razvoja.

Googleova linija PaLM-Gemini evoluira od gustog Pathways skaliranja prema redizajnu usmjerenom na učinkovitost s višejezičnim pretreniranjem, do nativne multimodalnosti s rijetkim usmjeravanjem i memorijski učinkovitom pažnjom za duge kontekste. PaLM je pokazao da se opći modeli trenirani na mješavini prirodnog jezika i znatnog koda učinkovito prenose na programerske zadatke. PaLM 2 je usavršio ravnotežu skaliranja i podataka. Gemini 1 i 1.5 uveli su nativnu multimodalnost, a Gemini 1.5 je dodao rijetke stručnjake, poboljšanja učinkovitosti

i kontekst od milijun tokena, omogućujući razumijevanje na razini repozitorija. Gemini 2 i 2.5 naglašavaju učinkovitost, rezoniranje i kodnu inteligenciju s proširenim kontekstom.

Anthropicova Claude linija evoluira od dekoder-samo LLM-ova usklađenih putem RLHF i ustavne umjetne inteligencije do alata obogaćenih dugim kontekstom i agentskih kodera. Claude 1 i 2 dodali su dugi kontekst i sigurnije praćenje uputa. Claude 3 i 3.5 uveli su nativnu multimodalnost i pozivanje funkcija s poboljšanjima na HumanEvalu. Claude 4 i 4.5 integriraju promišljeno rezoniranje i alatni lanac za korištenje računala, uključujući izolirani terminal, uređivač, upravitelj paketa i preglednik, s kontroliranim korištenjem alata i paralelnim računanjem u vrijeme testiranja.

Grok serija od xAI-ja evoluira od vlasničkog LLM-a koji prati upute u agentsku, kodu orijentiranu obitelj s duljim kontekstom. Generacija Grok-4 naglašava nativno korištenje alata i načine promišljanja s pretraživanjem u stvarnom vremenu, plus kodnu specijaliziranu krajnju točku.

2.2.2 Otvoreni modeli za kod

Zatvoreni modeli definirali su horizonte mogućeg, ali otvoreni modeli demokratizirali su pristup. Zamislite razliku između vlasničke tvornice lijekova koja drži formulu u sefu i znanstvenika koji objavljuju cijeli recept u akademskom časopisu. Oboje proizvode lijek, ali otvoreni pristup omogućuje drugima da ponove, prilagode i poboljšaju. Upravo to se dogodilo s otvorenim modelima za kod.

Razvoj otvorenih velikih jezičnih modela za kod može se sustavno kategorizirati u četiri evolucijske faze na temelju arhitektonskih inovacija i funkcionalnih sposobnosti.

Faza 1: Pretreniririrani enkoder modeli (2020-2021). Početna faza bila je obilježena enkoder modelima poput CodeBERTa Feng i ostali (2020), GraphCodeBERTa i CodeT5. Ovi modeli primarno su se fokusirali na zadatke razumijevanja koda, uspostavljajući temeljno poravnanje koda i teksta putem dvosmjernih mehanizama pažnje. Njihove ključne snage ležale su u klasifikaciji koda, otkrivanju ranjivosti i semantičkom pretraživanju koda.

Faza 2: Generativni modeli (2021-2022). Druga faza svjedočila je pojavi generativnih modela uključujući CodeT5 i CodeGPT, koji su uveli enkoder-dekoder arhitekture sposobne i za razumijevanje i za generiranje koda. CodeParrot je bila obitelj dekoder-samo GPT-2 modela specifično treniranih za generiranje Python koda. CodeGPT je bila obitelj GPT transformera razvijenih od strane Microsoft Researcha. T5 serija za kod koristila je tokenizator svjestan koda i pretreniranje svjesno identifikatora.

Faza 3: Veliki jezični modeli (2023-2024). Treća faza označila je paradigmatski pomak s pojavom otvorenih jezičnih modela velikog razmjera poput StarCodera R. Li i ostali (2023), Code LLaMA-e, DeepSeek-Codera i CodeQwena Hui i ostali (2024). Ovi modeli pokazali su izvanredne sposobnosti u složenom generiranju koda, višekružnom konverzacijskom programiranju i praćenju uputa, demonstrirajući da otvoreni modeli mogu postići konkurentne performanse s vlasničkim suparnicima. StarCoder i StarCoderBase su dekoder-samo modeli

s otvorenim pristupom Lozhkov i ostali (2024). Code LLaMA je obitelj bazirana na LLaMA 2 Touvron i ostali (2023) koja naglašava snažno popunjavanje, podršku za dugi kontekst i praćenje uputa za programiranje. DeepSeek-Coder je obitelj otvorenog koda (1.3B do 33B) trenirana od nule na višejezičnim bazama koda. StarCoder2 unapređuje BigCode liniju sa The Stack v2, većim i raznolikijim izvorom podataka. Qwen2.5-Coder proširuje se na obitelj od 0.5B do 32B.

Faza 4: Napredno skaliranje i agentski modeli (2024-2025). Trenutna faza predstavlja dva velika razvoja. Prvo, masivno skaliranje parametara kroz MoE arhitekture koje održavaju visoku učinkovitost zaključivanja. Drugo, evoluciju prema agentskim sustavima koji integriraju korištenje alata, višekorakno rezoniranje i sposobnosti interakcije s okruženjem. DeepSeek-Coder-V2 usvaja MoE okosnicu s mješovitim podatcima i proširenim kontekstom. Qwen3-Coder unapređuje agentske sposobnosti. GLM-4.5 i 4.6 su otvoreni MoE temeljni modeli za agentske, rezonantne i kodne zadatke. DeepSeek-V3 serija postiže vrhunske performanse otvorenog koda na kodnim mjerilima, rivalizirajući zatvorene modele.

> **Note**
>
> Za vašu profesionalnu praksu: Otvoreni modeli više nisu samo laboratorijski eksperimenti. Oni su produkcijski alati koji rade na pristupačnom hardveru, poštuju licencne uvjete i omogućuju organizacijama punu kontrolu nad kodom koji generiraju. Razumijevanje ove evolucije od enkoder modela do agentskih sustava ključno je za donošenje informirane odluke.

3 Zadaci, mjerila i evaluacija koda

Zamislite da ste učitelj u školi i da vaši učenici tvrde da savršeno vladaju matematikom. Kako ćete to provjeriti? Trebat će vam ispiti. Ne bilo kakvi ispiti, nego takvi koji pokrivaju aritmetiku, algebru, geometriju i analizu, koji imaju pitanja različite težine, i koji ne dopuštaju učenicima da odgovaraju na pitanja koja su već vidjeli tijekom vježbanja. Trebat će vam i jasni kriteriji ocjenjivanja, jer razlika između potpuno točnog odgovora, djelomično ispravnog postupka i potpuno pogrešnog pristupa mora biti transparentna. Upravo ta logika stoji iza čitavog ekosustava zadataka, mjerila (engl. *benchmarks*) i metrika evaluacije koji služe za procjenu sposobnosti velikih jezičnih modela za kod. Bez njih, tvrdnje o sposobnostima modela bile bi jednako vjerodostojne kao tvrdnje učenika koji kaže da zna matematiku ali odbija pisati ispit.

Krajolik kodnih zadataka i mjerila može se organizirati u hijerarhijsku taksonomiju duž tri razine granularnosti. Prva razina obuhvaća zadatke na razini izjave, funkcije i klase, uključujući dovršavanje koda, popunjavanje sredine (FIM), generiranje koda, uređivanje i popravljanje grešaka, učinkovitost koda, preferencije koda, rezoniranje i odgovaranje na pitanja, prevođenje koda i generiranje testnih slučajeva. Druga razina obuhvaća zadatke na razini

repozitorija, uključujući generiranje i dovršavanje u velikim bazama koda, domenski specifično generiranje koda, uređivanje, refaktoriranje i suradnju agenata, generiranje poruka predaja (engl. *commit messages*), rješavanje zadataka softverskog inženjerstva i sveobuhvatni razvoj softvera. Treća razina obuhvaća agentske sustave, uključujući korištenje alata, duboko istraživanje, web pretraživanje, interakciju s grafičkim korisničkim sučeljima i korištenje terminala. Zajedno, ove tri razine čine konsolidiranu mapu trenutnih evaluacijskih resursa kroz sve sposobnosti i razmjere.

3.1 Metrike evaluacije

3.1.1 Pass@k

Pass@k je temeljna metrika za procjenu sposobnosti modela da generira točne programerske rješenja. U ovoj metrici model generira k kandidatskih rješenja za svaki zadatak, a rješenje se smatra prolaznim ako je bar jedno od k rješenja točno, provjereno izvršavanjem na testnim slučajevima. Izbor vrijednosti k (obično 1, 5, 10 ili 100) odražava različite scenarije primjene. Pass@1 simulira okruženja gdje je dostupna samo jedninatačna rješenja, poput Chat sučelja, gdje korisnik očekuje jedan odgovor. Pass@100 oponaša scenarije offline obrade gdje se mogu generirati i rangirati mnoga rješenja prije isporuke krajnjem korisniku. Pass@k je posebno važna jer omogućuje mjerenje sposobnosti modela u scenarijima gdje je paralelnost i raznolikost generiranja praktično dostupna.

3.1.2 Točnost (Accuracy)

Točnost mjeri postotak testnih primjera na kojima je model dao točan odgovor. U domeni kodiranja, točnost je često definirano kao omjer broja testnih slučajeva koje je generirani kod prošao u odnosu na ukupan broj testnih slučajeva. Iako je jednostavna i intuitivna metrika, točnost može biti zljegledno ako testni slučajevi nisu reprezentativno pokrivanje mogućih scenarija. Primjerice, model s točnosti od 85% može sjajno raditi na jednostavnim operacijama, ali potpuno zakazati na rubnim slučajevima ili na kodu s vanjskim učincima.

3.1.3 BLEU i ROUGE

BLEU (engl. *Bilingual Evaluation Understudy*) i ROUGE (engl. *Recall-Oriented Understudy for Gisting Evaluation*) metriken su izvorno razvijene za procjenu strojnog prevođenja i sumarizacije teksta, ali su adaptirane i za procjenu generiranja koda. BLEU mjeri preklapanje između generiranog i referentnog koda na razini n-grama, dok ROUGE mjeri dohvat (engl. *recall*) koji se može prilagoditi za različite vrste kodnog podudaranja. Međutim, obje metriken imaju ograničenja u domeni kodiranja jer su osjetljive na sintaksne varijacije koje ne utječu na semantiku ili ispravnost koda. Kod je inherentno robustan na permutacije, preimenovanje

varijabli i razumnu preoblikovanje logike, a metriken kao BLEU i ROUGE ne mogu biti te razlike.

3.1.4 Tree-based Metrics

Tree-based metriken poput CodeBLEU izbjegavaju probleme s površinskim uspoređivanjem tako što prvo parsiraju kod u apstraktno sintaksno stablo (engl. *Abstract Syntax Tree* ili AST) i zatim mjere sličnost strukturalno, uključujući sličnost u nomenklaturi, ključnim riječima, AST čvorovima i redoslijedu putanje. CodeBLEU posebno je pokazao korelaciju sa ljudskom evaluacijom za razumijevanje ispravnosti koda, što ga čini bol im pokazateljem od površinskog BLEU za kodne zadatke. Ipak, čak i CodeBLEU ne može u potpunosti zarobiti semantičke varijacije koje su moguće u kodu, što znači da ga je kao jedinu metriku potrebno kombinirati s metodama koje izvršavaju kod.

3.1.5 Execution-based Metrics

Execution-based metriken mjere ispravnost koda izvršavanjem na testnim slučajevima. Pass@1, Pass@k, točnost, brzina izvršavanja i potrošnja memorije sve su execution-based metriken koje vam daju operativne povratne informacije o kodu koji je generiran. Prednosti su jasne: ako kod prođe sve testne slučajeve, model je dao ispravno rješenje. Nedostaci su praktični i teorijski. Praktični nedostaci uključuju potrebu za sigurnom izolacijom okruženja za izvršavanje, jer je mogućnost da generirani kod sadržava zlonamjerni kod ili beskonačne petlje, te potrebu za vremenskim ograničenjima i ograničenjima memorije. Teorijski nedostaci uključuju potencijal za curenje podataka ako su testni slučajevi dostupni tijekom pretreniranja, što je čini važno dekontaminirati evaluacijske skupove prije procjene.

3.1.6 Embedding-based Metrics

Embedding-based metriken koriste predstavljene modela za kodiranje (engl. *code embedding*) da mjere sličnost između referencirati i generiranog koda. Umjesto direktnog podudaranja ili izvršavanja, ove metriken koriste semantičko razumijevanje da procijene koliko su dva komada koda bliski u prostoru značenja. Primjer je CodeSearchNet Evaluation (CSE) koja koristi obuke enkoder-dekoder modele za generiranje ugrađivanja i mjerenje kosinusne sličnosti između ugrađivanja referencitanog i generiranog koda. Dok su ove metriken fleksibilne i mogu se primijeniti na bilo koji kod bez potrebe za izvršavanjem, njihova kvaliteta ovisna je o kvaliteti osnovnog modela za ugrađivanje i mogu biti osjetljive na promjene u samom modelu tijekom vremena.

3.1.7 Human Evaluation

Ljudska evaluacija je temeljna metrika koja se koristi za validaciju svih automatiziranih metrika. Ljudski prosuditelji procjenjuju ispravnost, čitljivost, učinkovitost i prikladnost generiranog koda prema različitim kriterijima. Ljudska evaluacija je skupna, ali omogućuje hvatanje nijansi koje automatizovane metriken mogu propustiti, kao što su idiomatski kod, kvaliteta komentara i prikladnost imena varijabli za čitljivost. Standardne prakse uključuju višestruko prosud - više ljudskih prosuditelja procjenjuje svaki primjer - i izračunavanje Cohen-ovog Kappa koeficijenta za procjenu dogovor između prosuditelja.

3.2 Zadaci na razini funkcije

HumanEval je reprezentativan benchmark za procjenu sposobnosti modela na zadacima na razini funkcije. Skup sadrži 164 ručno napisane funkcije s opise u prirodnom jeziku, testne slučajeve i očekivane izlaze. Za svaki zadatak, model mu je dana engleska opisna funkcija i trebao je generirati Python implementaciju. Evaluacija koristi Pass@k s različitim vrijednostima k kako bi se prosudila kako se sposobnost modela poboljšava sa višestrukim pokušajima. MBPP (engl. *Mostly Basic Programming Problems*) pruža 974 problem višestruke težine, od početne do srednje razine, što pruža šira pokrivanja. BabelCode proširuje zadatke na 23 jezika, omogućavajući procjenu sposobnosti modela bez jezika specifičnog biranja. LiveCodeBench koristi Codeforces kao izvor za na razini repozitorija probleme koji se kontinuirano ažuriraju kako se dodaju novi problemi, eliminirajući mogućnost data leakage i omogućavajući stalnu evaluaciju modela.

> **Note**
>
> Za vašu profesionalnu praksu: Razumijevanje različitih mjerila i metrika ključno je jer izbor mjerila može značajno utjecati na kako se modeli rangiraju i opredjeljuju. Neki modeli mogu biti odličan Pass@1 (prvi pokušaj), ali loši u Pass@100 (stotina pokušaja), što ima grote implikacije za kako ga implementirate u stvarnom okruženju.

4 Usklađivanje

Zamislite da ste upravo zaposlili sjajnog mladog inženjera koji je završio fakultet s najboljim ocjenama. Zna teoriju, razumije algoritme, može riješiti gotovo svaki zadatak na papiru. Ali prvog dana na poslu sjedne za računalo i počne pisati kod koji je tehnički ispravan, ali ne prati konvencije vašeg tima, ignorira sigurnosne prakse vaše organizacije i proizvodi rješenja koja nitko drugi ne može održavati. Što trebate napraviti? Trebate ga uskladiti, naučiti ga ne samo što je ispravno, nego i što je poželjno, korisno i sigurno u kontekstu stvarnog rada. Upravo to je usklađivanje (engl. *alignment*) u kontekstu velikih jezičnih modela za kod. To je

proces prilagodbe pretreniranih LLM-ova da prate ljudske upute i učinkovito obavljaju zadatke kodiranja.

Usklađivanje se oslanja na dva temeljna pristupa. Prvi je nadzirano fino ugađanje (engl. *Supervised Fine-Tuning* ili SFT), u kojem model uči iz označenih skupova podataka koji pokrivaju zadatke poput generiranja koda, popravljanja i prevođenja. Drugi je pojačano učenje (engl. *Reinforcement Learning* ili RL), koje koristi signale nagrade za daljnje usavršavanje ponašanja modela. Posebno moćna varijanta je pojačano učenje s provjerljivim nagradama (engl. *Reinforcement Learning with Verifiable Rewards* ili RLVR), gdje modeli primaju determinističke povratne informacije o prolazu ili padu od testnih slučajeva ili kompajlera, omogućujući im razvoj strukturiranog rezoniranja, samoverifikacije i sposobnosti ispravljanja grešaka. Zajedno, ove metode usklađivanja transformiraju opće pretreniranje modele u specijalizirane pomoćnike za kodiranje sposobne za razumijevanje zahtjeva, generiranje ispravnih rješenja i rukovanje složenim scenarijima razvoja softvera u stvarnom svijetu.

4.1 Nadzirano fino ugađanje (SFT)

Nadzirano fino ugađanje je proces treniranja pretreniranog jezičnog modela na označenom skupu podataka. Uobičajen i moćan način formatiranja tih podataka je ugađanje na uputama (engl. *instruction tuning*), gdje model uči pratiti naredbe na prirodnom jeziku. Opseg podataka za ugađanje kodnih uputa je širok, pokrivajući raznolik raspon programerskih zadataka uključujući, ali ne ograničavajući se na, generiranje koda, popravljanje koda i prevođenje koda.

Rani podatci za ugađanje kodnih uputa izvlačeni su iz raznih programerskih zajednica i platformi, poput parova kod/komentar iz GitHub repozitorija i korisničkih pitanja i odgovora sa StackExchangea. Međutim, budući da svaki podatak dolazi od različitih korisnika iz raznolikih izvora, kvaliteta je često neujednačena. Kao alternativa, pristup samoinstrukcije (engl. *self-instruct*) koristi moćnije LLM-ove za generiranje kvalitetnijih i standardiziranijih podataka kroz visokokvalitetne demonstracije i učenje iz konteksta.

4.1.1 Jednokružno nadzirano fino ugađanje

Kod u ranim skupovima podataka uglavnom se sastojao od osnovnih operacija poput stvaranja objekata i aritmetike, bez naprednih algoritama i složene logike. Da bi se to riješilo, predložena je metoda Evol-Instruct koja vodi model da generira složenije proširene podatke na temelju izvornih podataka koristeći niz ručno izrađenih heurističkih pravila. Kako se obujam generiranih podataka povećava, raste i vjerojatnost da metoda samoinstrukcije proizvede ponavljajuće ili slične podatke. Učinkovito rješenje je uvesti ljudski pisane kodne podatke tijekom procesa generiranja, koristeći njihovu bogatu raznolikost za izbjegavanje ponavljanja. Semi-Instruct metoda koristi raznolike podatke kao temelj i primjenjuje metodu samoinstrukcije za njihovo prepisivanje. OSS-Instruct usvaja sličan proces s razlikom što se sirovi podatci sastoje od isječaka koda umjesto potpunog koda.

4.1.2 Višekružno nadzirano fino ugađanje

Ključna značajka koja razlikuje kod od prirodnog jezika je njegova izvršivost. Stoga se generirani kod može verificirati korištenjem kompajlera i interpretera, pružajući jeftine i visoko učinkovite signale povratnih informacija bez ljudske intervencije za vođenje poboljšanja nakon pojave grešaka. Zamislite to kao kuhara koji ne treba čekati gosta da mu kaže je li jelo dobro, nego može sam kušati svaki korak pripreme.

AIEV-Instruct postavlja agenta ispitivača i agenta programera temeljenih na istom velikom modelu. Kada kod koji generira agent programer ne prođe verifikaciju, agent ispitivač intervenira, postavljajući pitanja na temelju greške kako bi olakšao poboljšanje. Podatci konstruirani s povratnim informacijama inherentno su višekružni, dok korisnici u praktičnim primjenama općenito preferiraju jednokružne interakcije kad god je to moguće. Za premošćivanje ovog jaza između treniranja i zaključivanja, predložena je metoda samodestilacije koja dodaje sažetak poput završnog kruga na kraju višekružnih podataka.

4.1.3 SFT za zadatke na razini repozitorija

Dok se tradicionalno ugađanje kodnih uputa fokusiralo na izolirane funkcije ili zadatke na razini datoteka, složenost razvoja softvera u stvarnom svijetu zahtijeva modele koji mogu razumjeti i navigirati čitavim strukturama repozitorija. Skupovi podataka za SFT na razini repozitorija pojavili su se kao važan resurs za treniranje kodnih LLM-ova za rukovanje međudatotečnim ovisnostima, višedatotečnim uređivanjima i dugokontekstnim rezoniranjem koji karakteriziraju praktične scenarije softverskog inženjerstva.

SWE-smith uvodi skalabilni cjevovod koji generira veliku kolekciju instanci zadataka iz GitHub repozitorija, predstavljajući red veličine povećanja nad prethodnim radovima. SWE-Synth nadopunjuje ovo sintetiziranjem provjerljivih podataka o popravcima grešaka kroz radne procese uklanjanja grešaka vođene LLM-ovima. SWE-Gym pruža Python instance zadataka s izvršivim okruženjima za izvršavanje, omogućujući pristupe pojačanog učenja. RepoBench evaluira automatsko dovršavanje kroz tri međusobno povezana zadatka kroz desetke tisuća Python i Java repozitorija, specifično mjereći razumijevanje međudatotečnog konteksta.

Unatoč izazovima poput računalnih troškova održavanja okruženja za izvršavanje i osiguranja kvalitete podataka tijekom skaliranja, dosljedna poboljšanja kroz mjerila i jasni zakoni skaliranja podataka ukazuju da je ulaganje u veće, raznolikije skupove podataka na razini repozitorija obećavajući put za unapređenje autonomnih programerskih sustava.

4.1.4 Metode bazirane na rezoniranju

LLM-ovi bazirani na rezoniranju predstavljaju značajan napredak nad tradicionalnim modelima ugođenim na uputama uključivanjem eksplicitnih procesa lanca misli (engl. Chain-of-Thought ili CoT) tijekom zaključivanja, omogućujući im da dekompoziraju složene probleme u međukorake

i provode sustavniju analizu. Zamislite razliku između studenta koji pokušava odmah napisati konačni odgovor na ispitu i studenta koji prvo napiše plan, zatim korak po korak izvodi rješenje, provjerava svaki korak i tek onda zapisuje konačni rezultat. Drugi student je pouzdaniji, čak i kada je problem složen, jer je naučio proces razmišljanja, a ne samo konačne odgovore.

Mehanizam iza uspjeha CoT-a vezan je za način na koji strukturira proces rješavanja problema za model. Generiranje eksplicitnog lanca rezoniranja služi kao računalna bilježnica (engl. scratchpad), omogućujući modelu da preuzme međukorake i zaključke u svoj kontekstni prozor. Zanimljivo, učinkovitost ovog procesa čini se da više ovisi o strukturi rezoniranja nego o činjeničnoj ispravnosti njegovog sadržaja. Studije su pokazale da modeli mogu naučiti učinkovita ponašanja rezoniranja čak i kada su fino ugođeni na CoT demonstracijama s pogrešnom logikom ili netočnim konačnim odgovorima, što sugerira da je primarna uloga CoT-a tijekom finog ugađanja naučiti model strukturiran, promišljen pristup rješavanju problema.

4.1.5 Strategije treniranja

Nakon proširenja podataka, neki radovi fokusirali su se na učinkovitije treniranje. Jedan aspekt je filtriranje kvalitete velikih skupova podataka. Pristupi uključuju grupiranje svih podataka i zadržavanje različitog broja podatkovnih točaka iz svake grupe na temelju gustoće centra grupe, čime se postiže deduplikacija podataka. Drugi pristupi najprije se fokusiraju na težinu i ispravnost podataka prije razmatranja raznolikosti, trenirajući bodovatelja složenosti za odabir najtežih podataka i generator testova za izračun stope prolaza testnih slučajeva za svaku podatkovnu točku.

Osim kvalitete podataka, višezadaćna ravnoteža également je važna. Tretiranje generiranja koda i evaluacije koda kao dvofaznog zadatka treniranja značajno poboljšava performanse modela. Integracija višestrukih funkcija gubitka i predlaganje okvira za višezadaćno fino ugađanje koji može fino ugoditi zadatke paralelno pokazali su se konzistentno korisnim.

4.1.6 Izazovi u SFT-u

Unatoč brojnim metodama i skupovima podataka koji su se pojavili u ugađanju kodnih uputa, nekoliko izazova i dalje ostaje. Curenje podataka znači da trenirni podatci mogu sadržavati informacije iz testnih ili evaluacijskih skupova, što ne samo da dovodi do napuhanih performansi nego može prikriti nedostatke modela u scenarijima iz stvarnog svijeta. Pristranost podataka očituje se u pretjeranom fokusu na jednostavne programerske zadatke, što dovodi do loših performansi modela na složenim zadacima.

Nedovoljna višejezična podrška znači da su trenutni skupovi podataka pretežno bazirani na uskom skupu popularnih programskih jezika poput Pythona ili JavaScripta, ograničavajući primjenjivost modela za programere koji rade u drugim jezicima poput C++, Rusta, Goa

ili Swifta. Jezici se fundamentalno razlikuju ne samo u površinskoj sintaksi, nego i u dubljim semantičkim aspektima poput idioma, standardnih biblioteka, upravljanja memorijom i dizajnerskih obrazaca.

4.2 Hladni start i destilacija podataka rezoniranja

Ovaj odjeljak bavi se pitanjem kako od nule konstruirati visokokvalitetne podatke za fino ugađanje rezoniranja. Zamislite da trenirate mladu šahistkinju. Ne dajete joj tisuće nasumičnih partija, nego pažljivo birate partije velikih majstora koje demonstriraju važne strategije. Upravo to rade istraživači kada konstruiraju skupove podataka za destilaciju rezoniranja.

Proces se odvija u šest koraka. Prvo, prikupljanje izvora, nabavka sirovih problema i materijala koji čine temeljni sloj skupa podataka, iz akademskih natjecanja, online foruma i sintetičkih ili destiliranih skupova podataka. Drugo, čišćenje i dekontaminacija podataka za poboljšanje integriteta i pouzdanosti trenirnih podataka, uključujući semantičke usporedbe putem LLM-sudca za identificiranje ne samo točnih duplikata nego i parafraziranih problema. Treće, filtriranje pitanja i procjena kvalitete i težine za odabir podskupa problema visoke poluge iz većeg bazena. Četvrto, generiranje lanaca rezoniranja, stvaranje detaljnih rješenja korak po korak za svaki problem. Peto, filtriranje i usavršavanje rješenja za osiguranje da generirana rješenja nisu samo ispravna nego i visokokvalitetna. Šesto, konačna konstrukcija skupa podataka, kompilacija svih obrađenih problema i njihovih usavršenih rješenja u koherentan format spreman za fino ugađanje modela.

Pristupi konstruiranju konačnog skupa podataka variraju primarno u svom razmjeru i strateškom obrazloženju. Neki projekti poput LIMO-a sa samo 800 primjera i s1 sa samo 1.000 uzoraka prioritiziraju stvaranje izuzetno malih ali visokokvalitetnih skupova podataka, dokazujući da opsežni podatci nisu uvijek nužni za postizanje snažnih sposobnosti rezoniranja. Drugi projekti, ciljajući na vrhunske performanse, konstruiraju znatno veće skupove, poput OpenThoughts3 s 1,2 milijuna uzoraka ili OpenMathReasoning s preko 500.000 jedinstvenih problema. Optimalni pristup odražava jezgrenu hipotezu istraživačkog projekta, s nekim istraživačima koji prioritiziraju podatkovnu učinkovitost i drugima koji se fokusiraju na maksimiziranje performansi i svestranosti.

4.3 Višejezično razumijevanje i generiranje koda

Zamislite prevoditelja koji savršeno prevodi s engleskog na francuski, ali potpuno zakazuje kada mu date tekst na japanskom. Upravo to je situacija s mnogim kodnim LLM-ovima koji briljiraju u Pythonu ali jedva funkcioniraju u Rustu ili Fortranu. Razvoj višejezičnih kodnih LLM-ova može se podijeliti u faze.

Rani modeli od 2020. do 2021. godine poput JavaBERTa i C-BERTa primarno su se fokusirali na pojedinačne programske jezike s relativno malim skalama parametara od 125 do 350 milijuna. Srednje razdoblje od 2022. do 2023. godine svjedočilo je probojnom napretku, s višejezičnim

modelima poput Codexa i AlphaCodea koji su se proširili na 12 do 41 milijardu parametara i podržavali 6 do 12 glavnih programskih jezika. Najnovija generacija modela od 2024. do 2025. godine postigla je skok. DeepSeek-Coder-V2 dosegnuo je 236 milijardi parametara s podrškom za 338 jezika, dok StarCoder2 podržava više od 600 jezika treniranih na više od 4 terabajta podataka, označavajući ulazak kodne inteligencije u eru ultra-velikog višejezičja.

Trenirni korpusi evoluirali su od pojedinačnih GitHub repozitorija do ultra-velikog fuziranja iz višestrukih izvora, uključujući GitHub, Stack Overflow, CodeSearchNet, seriju The Stack, The Pile, sintetičke udžbeničke podatke, tragove izvršavanja i paralelne prevodilačke korpuse. Arhitektonski, područje pokazuje raznolikost od BERT enkoder modela, preko T5 enkoder-dekoder modela, do GPT dekoder modela i pristupa grafičkih neuronskih mreža koji integriraju strukturu koda. Najnoviji trendovi uključuju male učinkovite modele koji traže ravnotežu između performansi i troška, međujezično prevođenje koda koje se pojavljuje kao neovisni istraživački smjer, multimodalnu fuziju i specijaliziranu optimizaciju za niskoresursne i naslijeđene jezike.

4.4 Multimodalno razumijevanje i generiranje koda

S brzim razvojem vizualno-jezičnih multimodalnih velikih jezičnih modela, istraživanje generiranja koda proširuje se od čisto tekstualnog ulaza prema novoj paradigmi koja uključuje više modaliteta poput slika, skica i interaktivnih signala. Multimodalno generiranje koda (engl. Multimodal Code Generation ili MCG) cilja omogućiti modelima da razumiju vizualne dizajnerske namjere i generiraju izvršivi, renderirajući visokokvalitetan kod, čime premošćuje jaz između apstraktnog razmišljanja visoke razine i implementacije koda niske razine. Na temelju scenarija primjene i tehničkog fokusa, ovo područje dijeli se u tri osnovna podpolja.

4.4.1 Generiranje korisničkog sučelja

Evolucija je tekla od jednog modaliteta prema višemodalnosti i od statičnog prema dinamičnom. Rani sustavi poput pix2codea demonstrirali su izvedivost mapiranja snimaka zaslona grafičkog sučelja u platformski specifičan kod. Potom su uslijedili pristupi slika u kod, dizajn u kod, skica u kod i interakcija u kod.

Design2Code je izgradio opsežno mjerilo koje sadrži 43.000 parova snimka zaslona ili HTML-a, sustavno evaluirajući 9 multimodalnih velikih jezičnih modela i predlažući hijerarhijske metrike evaluacije. Sketch2Code je objavio prvo mjerilo s 731 visokokvalitetnom rukom nacrtanom skicom. Interaction2Code je definirao novu paradigmu zadataka za dinamičnu razinu, objavljujući mjerilo sa 127 web stranica, 374 interakcije i 31 vrstu događaja.

4.4.2 Web utjelovljena inteligencija

Ovo područje nadilazi opseg statičnog generiranja koda, zahtijevajući od AI agenata da dovršavaju složene zadatke u stvarnim web okruženjima kroz petlje promatranja, rezoniranja i djelovanja. WebArena pruža realistične i složene otvorene platforme s funkcionalnim okruženjima repliciranim sa stvarnih web stranica i više od 800 zadataka dugoročnog planiranja.

WebVoyager je prekretnica u ovom području, prvi put ostvarujući cjelovite multimodalne web agente koji izravno koriste snimke zaslona kao ulaz za simulaciju ljudskog ponašanja pregledavanja. Agent-E predlaže hijerarhijske arhitekture agenata, dekomponirajući složene web zadatke u planera i navigatora preglednika.

4.4.3 Generiranje artefakata softverskog inženjerstva

Ovo podpolje cilja stvaranje pratećih artefakata za kod, uključujući UML dijagrame, dijagrame podataka, dijagrame toka, dijagrame arhitekture i druge vizualne dokumente softverskog inženjerstva. nvAgent predlaže višeagentski suradnički sustav za generiranje vizualizacija iz prirodnog jezika. Plot2Code uspostavlja sveobuhvatno mjerilo za evaluaciju multimodalnih LLM-ova u generiranju koda iz znanstvenih grafova. Flow2Code evaluira sposobnosti mapiranja dijagrama toka u kod, pokrivajući 15 programskih jezika. ArtifactsBench predlaže probojnu automatiziranu multimodalnu paradigmu evaluacije, rješavajući bolnu točku tradicionalne evaluacije koja ignorira vizualnu vjernost i potpunost interakcije.

4.5 Pojačano učenje u kodnoj inteligenciji

Pojačano učenje transformiralo je pristup treniranju modela za kodiranje na način koji je teško podcijeniti. Zamislite da učite dijete igrati šah. Nadzirano fino ugađanje je kao kad mu pokazujete partije velemajstora. Pojačano učenje je kao kad dijete samo igra partije, dobiva nagradu kad pobijedi i kaže mu se samo rezultat. Kroz tisuće partija, dijete razvija intuiciju. Upravo to rade algoritmi pojačanog učenja za kodne LLM-ove, koriste provjerljive nagrade poput prolaza jediničnih testova, grešaka kompajlera i povratnih informacija statičke analize.

4.5.1 Algoritmi pojačanog učenja

Metode proksimalne optimizacije politike (PPO) Schulman i ostali (2017) omogućuju online istraživanje s KL-regulariziranim ažuriranjima politike koja balansiraju raznolikost nasuprot vjernosti nadziranom prioru, često kombinirane s nagradama vođenim izvršavanjem i petljama popravaka za pojačanje stope prolaza i smanjenje halucinacija. Izravna optimizacija preferencija (DPO) Rafailov i ostali (2023) zaobilazi potrebu za online uzorkovanjem učeći iz parova preferencija destiliranih iz uzoraka ocijenjenih izvršavanjem (uspjeh nasuprot neuspjehu),

dajući stabilno usklađivanje za stil koda i korak po korak rezoniranje bez varijance procjena gradijenata politike.

Pojačano učenje iz povratnih informacija umjetne inteligencije (RLAIF) dodatno koristi jače modele verifikatora i kritičara za pružanje gustih, strukturiranih signala, omogućujući samoverifikaciju i iterativno usavršavanje. Grupna relativna optimizacija politike (GRPO) Shao i ostali (2024) izbjegava učenje vrijednosti uzorkujući G izlaza za upit i izračunavajući relativne prednosti, dajući stabilna ažuriranja bez kritičara i smanjene memorijske zahtjeve. DAPO poboljšava PPO ili GRPO s inženjerskim usavršavanjima uključujući Clip-Higher za održavanje entropije, dinamičko uzorkovanje, gubitak na razini tokena i oblikovanje nagrade za preduge izlaze.

4.5.2 RL za generiranje koda

CodeRL je bio pionir u primjeni PPO pristupa tretirajući LM koji generira kod kao mrežu aktera s mrežom kritičara koja pruža guste signale povratnih informacija. Područje se potom pomaknulo prema metodama izravne optimizacije preferencija. CodeDPO uvodi mehanizam samogeneriranja i validacije s algoritmima inspiriranim PageRankom za stvaranje skupova podataka preferencija koji optimiziraju ispravnost i učinkovitost. Najnovije, RLVR je nastao kao probojni pristup koji znatno pojačava sposobnosti generiranja koda. DeepCoder pokazuje učinkovitost ovih metoda, postižući izvanredne performanse na LiveCodeBenchu s distribuiranim RL treningom i dostižući razinu o3-mini s samo 14 milijardi parametara.

Za dovršavanje koda, RLCoder uvodi RL okvir za dovršavanje na razini repozitorija koji uči dohvatiti koristan sadržaj bez označenih podataka. Za prevođenje koda, CoTran je pionir pojačanog učenja s povratnim informacijama kompajlera i simboličkog izvršavanja za Java-Python prevođenje.

4.5.3 RL za razumijevanje koda

Sažimanje koda profitira od pojačanog učenja za rješavanje problema pristranosti izlaganja inherentnog tradicionalnim okvirima enkodera i dekodera. HAN proširuje ovaj pristup hijerarhijskim mehanizmima pažnje koji integriraju više značajki koda uključujući tipom obogaćena apstraktna sintaksna stabla i tokove kontrole. Za generiranje komentara, TAG uvodi okvir vođenja pomoćnim tipovima koji tretira izvorni kod kao N-arno stablo s informacijama o tipovima, koristeći hijerarhijsko pojačano učenje. Za pretraživanje koda, CoaCor usvaja novu perspektivu treniranjem modela anotacije koda za generiranje opisa koji poboljšavaju performanse dohvata koda.

4.5.4 RL za softversko inženjerstvo

Rješavanje problema u softverskim repozitorijima složen je izazov u kojem je RL pokazao izvanredan uspjeh. SWE-RL unapređuje rezoniranje LLM-ova kroz pojačano učenje na repozitorijima otvorenog koda. DeepSWE skalira RL treniranje za stvaranje potpuno otvorenog agenta za kodiranje koji postiže vrhunske performanse na SWE-benchu. Za refaktoriranje koda, Refactor-RL fino ugađa modele sekvenca u sekvencu i usklađuje ih korištenjem PPO-a, koristeći uspjeh kompilacije koda i prisutnost refaktoriranja kao signale nagrade. Za pregled koda, PPO-bazirani pristupi formaliziraju predikciju ishoda zahtjeva za spajanje kao Markovljeve procese odlučivanja sa specijaliziranim funkcijama nagrade. Za optimizaciju koda, Pearl uvodi okvir dubokog RL-a koji trenira agente za odabir optimalnih sekvenci poliedarskih transformacija.

4.5.5 RL za sigurnost koda

Otkrivanje grešaka u kompajlerima i softverskim sustavima predstavlja značajne izazove. RecBi je pionir pojačanog učenja za izolaciju grešaka kompajlera proširenjem tradicionalnih operatora mutacije strukturnim operatorima. Za automatski popravak programa, RePair uvodi nadzor baziran na procesu, koristeći model nagrađivanja kao kritičara za iterativnu optimizaciju politika popravaka. Za generiranje testova, CURE predlaže okvir ko-evolucije u kojem se sposobnosti kodiranja i generiranja jediničnih testova poboljšavaju kroz njihove ishode interakcije bez nadzora temeljne istine. Za fuzziranje (engl. fuzzing), CovRL-Fuzz kombinira LLM-ove s pojačanim učenjem vođenim pokrivanjem za testiranje JavaScript interpretera, uspješno identificirajući 58 sigurnosnih grešaka iz stvarnog svijeta uključujući 50 prethodno nepoznatih.

4.6 Pojačano učenje s provjerljivim nagradama

Pojačano učenje s provjerljivim nagradama (RLVR) nova je paradigma dizajnirana za pojačavanje sposobnosti rezoniranja u LLM-ovima, posebno za domene poput matematike i sinteze programa. Zamislite to ovako. U klasičnom pojačanom učenju, nagrada dolazi od ljudskog evaluatora koji kaže ovaj odgovor mi se sviđa ili ne sviđa, što je neizbježno subjektivno i skupo. U RLVR-u, nagrada dolazi od determinističkog verifikatora koji kaže kod prolazi sve testove ili kod ne prolazi testove. Nema subjektivnosti, nema ambigviteta, nema troška ljudske evaluacije. Umjesto oslanjanja na naučene modele nagrađivanja ili heurističke ocjene preferencija, funkcija provjerljive nagrade izravno mjeri ispravnost zadatka, primjerice, prolazi li generirani matematički dokaz do ispravnog odgovora ili prolazi li dio koda sve jedinične testove. Nagrada je tipično binarna (prolaz ili pad), pružajući jasan i pouzdan signal koji se precizno usklađuje s ciljevima zadatka.

Nedavno uvođenje GRPO algoritma i njegova primjena velikog razmjera u DeepSeek-R1 DeepSeek-AI (2025) doveli su RLVR u prvi plan istraživanja rezonantnih LLM-ova. GRPO modificira standardni PPO okvir izračunavanjem procjena prednosti unutar grupe generiranih

uzoraka umjesto treniranja zasebnog modela vrijednosti. Ovaj pristup grupne normalizacije eliminira potrebu za naučenom mrežom kritičara, značajno poboljšavajući učinkovitost i stabilnost treniranja uz održavanje usklađenosti s provjerljivom nagradom. Ova svojstva čine RLVR posebno prikladnim za domene intenzivnog rezoniranja poput algoritmičkog rješavanja problema i sinteze koda, gdje se ispravnost može eksplicitno provjeriti računalnim proročištem.

4.6.1 Skupovi podataka prikladni za RLVR u kodnim zadacima

Razvijena je raznolikost specijaliziranih kodnih skupova podataka za olakšavanje RLVR treniranja. Ovi skupovi podataka konstruiraju programerske probleme zajedno s determinističkim verifikatorima poput ulazno-izlaznih testnih slučajeva ili jediničnih testova, osiguravajući da se izlazi modela mogu automatski provjeriti za ispravnost.

CodeContests je velik skup natjecateljskog programiranja s približno 13.300 problema za treniranje AlphaCode modela. TACO agregira zadatke s natjecateljskih platformi pokrivajući uobičajene teme poput sortiranja, dinamičkog programiranja i algoritama na grafovima. Eurus-2-RL je mješoviti skup za RL treniranje s 27.000 kodnih problema svi opremljeni verifikatorima. ACECode-87K je sintetički skup od otprilike 87.000 problema kreiranih putem automatiziranog generiranja testnih slučajeva. SYNTHETIC-1 je masivni skup s preko 1,4 milijuna zadataka koji obuhvaćaju matematiku, programiranje i softversko inženjerstvo. KodCode je najveći potpuno sintetički skup kodnih problema do danas s oko 48.400 trojki pitanje ili rješenje ili test. HardTests je skup natjecateljskog programiranja obogaćen visokokvalitetnim sintetiziranim testnim slučajevima za smanjenje lažno pozitivnih rezultata. Code-R1-12K je kurirani skup od 12.000 instanci kodnih problema sklopljen kao dio otvorenog Code-R1 projekta.

Za vašu profesionalnu praksu, razumijevanje RLVR-a i ovih skupova podataka važno je iz jednog važnog razloga. Ova paradigma fundamentalno mijenja način na koji modeli uče kodirati. Umjesto da memoriraju obrasce iz trenirnih podataka, modeli razvijaju sposobnost rezoniranja, samoverifikacije i ispravljanja grešaka, sposobnosti koje se prenose na nove, neviđene probleme. Kada evaluirate model za kod, pitajte je li treniran samo sa SFT-om ili i s RLVR-om. Odgovor vam može reći koliko duboko model zaista razumije programiranje nasuprot tome koliko dobro reproducira obrasce koje je vidio tijekom treniranja. To je razlika između učenika koji je zapamtio odgovore i učenika koji je naučio razmišljati.

> **Note**
>
> Za vašu profesionalnu praksu: Kodovi koji dolaze iz modela treniranog s RLVR-om strukturirano su drugačiji od onih iz SFT-samo modela. RLVR modeli razvijaju eksplicitne stratege razmišljanja, provjeravaju svoju logiku tijekom generiranja i ispravljaju se kada detektiraju greške, što su vještine koje se transferiraju na nove, neviđene scenarije. To je razlika između memoriranja i razumijevanja.

5 Agenti za softversko inženjerstvo

Do sada smo govorili o modelima kao o alatima, daješ im zadatak, oni vrate odgovor. Ali što se dogodi kada model prestane biti alat i postane kolega? Zamislite da u vašem timu sjedi novi član koji ne samo da piše kod, nego i čita specifikacije, postavlja pitanja o zahtjevima, predlaže arhitekturu, piše testove, analizira zapisnike grešaka, pregledava tuđi kod i sudjeluje u planiranju sprinteva. Upravo to su agenti za softversko inženjerstvo (engl. *SWE Agents*), autonomni ili poluautonomni sustavi dizajnirani za podršku, automatizaciju ili poboljšanje tradicionalnih radnih procesa softverskog inženjerstva.

Nedavni razvoji u velikim jezičnim modelima doprinijeli su pojavi ovih agenata, koji demonstriraju potencijal za učinkovitost, točnost i skalabilnost kroz čitav životni ciklus razvoja softvera Jimenez i ostali (2024). Da bismo sustavno ispitali njihove sposobnosti i implikacije, SWE agente organiziramo prema njihovim zadacima unutar klasičnih faza vodopadnog modela, od analize zahtjeva i dizajna, preko razvoja softvera i testiranja, do postavljanja i održavanja.

5.1 Agenti kroz životni ciklus softverskog inženjerstva

5.1.1 Inženjerstvo zahtjeva

Inženjerstvo zahtjeva čini ključnu disciplinu u softverskom inženjerstvu koja premošćuje zahtjeve dionika s tehničkim implementacijama, osiguravajući da isporučeni sustav ispunjava korisnička očekivanja. Zamislite to kao razgovor između arhitekta i obitelji koja želi sagraditi kuću. Obitelj kaže želimo puno svjetla i otvoreni plan, ali arhitekt mora te nejasne želje pretvoriti u precizne nacrte s dimenzijama, materijalima i instalacijama. Upravo to rade agenti u fazi zahtjeva.

Prikupljanje. Simulacija korisnika putem agenata pojavila se kao moćna paradigma. Elicitron generira raznolike simulirane korisničke agente koji autonomno interagiraju s ciljnim proizvodima, bilježeći svoja iskustva i pružajući strukturirane povratne informacije.

Ispitivanje i usklađivanje. Nakon prikupljanja, faza ispitivanja osigurava da su prikupljeni zahtjevi koherentni, izvedivi i razumljivi. Višeagentski sustavi pokazali su se učinkovitima u automatizaciji ovog analitičkog procesa. Jedan pristup simulira virtualni razvojni tim sastavljen od agenata s određenim ulogama koji suradnički generiraju, evaluiraju i rangiraju korisničke priče.

Modeliranje i formalizacija. U ovoj fazi zahtjevi se prevode u strukturirane, strojno interpretabilne reprezentacije od apstraktnih UML modela do konkretnih prototipova korisničkog sučelja. PrototypeFlow koordinira više specijaliziranih dizajnerskih agenata pod nadzornim koordinatorom. DCGen koristi multimodalne agentske kolektive koji interpretiraju snimke zaslona i generiraju funkcionalni kod.

5.1.2 Razvoj softvera

Sinteza programa. Agenti za sintezu programa proširuju se izvan statičkog poticanja uključivanjem višekoračnog rezoniranja, verifikacije bazirane na testovima i petlji usavršavanja. Zamislite programera koji ne piše kod u jednom potezu, nego planira, piše, testira, pronalazi greške, popravlja ih i ponovo testira. Upravo taj ciklus simuliraju agenti za sintezu programa. AlphaCodium BUAA-SKLCCSE, Alibaba, ByteDance, M-A-P, BJTU, OPPO, HKUST (GZ), BUPT, TeleAI, Shanghai AI Lab, Manchester, StepFun, UoS, SCU, CASIA, NJU, Kuaishou, HIT, Huawei Cloud, Tencent, Monash/CSIRO, NTU, ZJU, BIT, Ubiquant, NUS, HNU, PKU, CSU (2025) potiče model da nacrta implementacije, analizira potencijalne nedostatke i regenerira poboljšana rješenja. MapCoder koordinira dohvatitelja, planera, kodera i uklanjatelja grešaka. ChatDev Qian i ostali (2024) i MetaGPT Hong i ostali (2024) proširuju ovu ideju simuliranjem višeuloških organizacija.

Tekst u SQL. Pretvaranje teksta u SQL igra kritičnu ulogu tijekom faze razvoja, omogućujući inteligentnim sustavima transformaciju zahtjeva na prirodnom jeziku u strukturirane upite baze podataka. DAIL-SQL razmatra i pitanja i SQL upite za odabir primjera. MAC-SQL i MCS-SQL usmjereni su na višeagentsku suradnju.

Generiranje komentara i pregled koda. Rani pristupi koristili su metode bazirane na mapiranju. Neuronski sekvenca-u-sekvencu modeli označili su pomak prema generativnim pristupima. Za automatizirani pregled koda, napredni agentski okviri uvode višeagentsku orkestraciju gdje specijalizirani agenti pregledavatelji simuliraju suradničke ljudske timove. Hydra-Reviewer uvodi paralelnu arhitekturu gdje se svaki agent fokusira na jednu dimenziju pregleda.

5.1.3 Testiranje softvera

Testiranje softvera je faza u kojoj se provjerava radi li sustav onako kako je zamišljen. Zamislite inspektora koji dolazi provjeriti novosagrađenu kuću. Ne provjerava samo izgleda li kuća lijepo, nego pritišće svaki prekidač, otvara svaku slavinu, penje se na svaki kat i namjerno pokušava pronaći ono što ne radi. Agenti za testiranje rade upravo to. ChatTester i ChatUniTest generiraju jedinične testove putem konverzacijskog pristupa. TestPilot automatizira generiranje testova za JavaScript projekte. CoverUp koristi iterativnu strategiju za poboljšanje pokrivanja koda testovima. Za fuzz testiranje, automatizirane tehnike koriste pojačano učenje. CKGFuzzer koristi grafove znanja za vođenje fuzz testiranja.

5.1.4 Održavanje softvera

Održavanje softvera obuhvaća aktivnosti nakon isporuke, uključujući lokalizaciju grešaka, generiranje zakrpi, rješavanje problema, analizu zapisnika, optimizaciju kompajlera, dekompilaciju i deobfuskaciju. AgentFL koristi višeagentski sustav koji simulira suradnički tim za uklanjanje

grešaka. Za generiranje zakrpi, RepairAgent uvodi potpuno automatiziranoga agenta koji autonomno izvodi lokalizaciju grešaka, generiranje zakrpi i validaciju u jedinstvenoj petlji povratnih informacija. AutoCodeRover kombinira navigaciju apstraktnog sintaksnog stabla s analitičkim rezoniranjem.

5.2 Opći kodovi agenti u softverskom inženjerstvu

Uz specijalizirane agente za pojedine faze, postoji čitava kategorija općih agenata koji funkcioniraju kao svestrani alati primjenjivi kroz više faza razvoja. OpenHands X. Wang i ostali (2024) pruža okvir s više podagenata za navigaciju, uređivanje i testiranje unutar izoliranog IDE-a. HyperAgent koristi četveročlani tim koji postiže vrhunske performanse na RepoBenchu alociranjem različitih modela za različite funkcije. SWE-Agent Yang i ostali (2024) pruža minimalističko ali učinkovito sučelje koje omugućuje LLM-u navigaciju repozitorijima, uređivanje datoteka i izvršavanje naredbi u terminalnom okruženju. AGENTLESS demonstrira da čak i pristup bez agenata može postići konkurentne rezultate.

5.3 Tehnike treniranja za SWE agente

Fino ugađanje. Nadzirano fino ugađanje koristi kurirane parove ulaz/izlaz poput parova problem/zakrpa za učenje iz demonstracija. Ključni resursi uključuju SWE-smith koji pruža skalabilni cjevovod za generiranje instanci zadataka, SWE-Gym koji pruža Python instance s izvršivim okruženjima i RepoBench koji evaluira dovršavanje kroz tisuće repozitorija.

Pojačano učenje. RL pruža zasebnu paradigmu pomičući fokus s oponašanja statičkih skupova podataka na učenje optimalnih ponašanja kroz izravnu interakciju s okruženjem. Metode gradijenata politike izravno optimiziraju politiku odlučivanja. Izvanmrežno pojačano učenje uči iz fiksnog, unaprijed prikupljenog skupa interakcija. Metode bazirane na preferencijama poput RLHF-a optimalne su kada signal nagrade nije lako definiran. Ključna spoznaja je da uspješna primjena RL-a u SWE agentima rijetko je samostalni proces. Gotovo uvijek mu prethodi SFT koji služi kao kritična faza inicijalizacije.

5.4 Budući trendovi SWE agenata

Razvoj SWE agenata označava jasnu tehnološku putanju od automatizacije specifične za zadatke prema autonomnijim i integriranijim sustavima koji obuhvaćaju čitav životni ciklus razvoja softvera.

Od specijaliziranih agenata do orkestracije čitavog životnog ciklusa. Trenutni SWE agenti često djeluju u neovisnim fazama. Značajan budući smjer bit će razvoj integriranih agentskih okvira za čitav životni ciklus koji orkestriraju radne procese koji se bešavno prelijevaju iz jedne faze u sljedeću.

Duboko kontekstualno razumijevanje i dugoročna memorija. Trajni izazov za SWE agente je ograničeni kontekstni prozor LLM-ova. Budućnost leži u agentima koji mogu graditi i održavati trajne, dinamičke grafove znanja softverskih projekata.

Od suradnje do samoevolucije. Buduća istraživanja pomiču se od jednoagentskih sustava prema suradničkim višeagentskim ekosustavima. Gledajući dalje, agenti će steći sposobnost samoevolucije.

Sinergistička suradnja čovjeka i agenta. Najutjecajnija budućnost vjerojatno uključuje sinergističku suradnju čovjeka i agenta, a ne potpunu autonomiju. Umjesto da zamjenjuju programere, agenti će evoluirati u proaktivne inteligentne programere u paru.

Povjerenje, sigurnost i provjerljivost po dizajnu. Kako SWE agenti stječu više autonomije i dobivaju veći pristup produkcijskim sustavima, osiguravanje da su njihove akcije sigurne i provjerljive postaje ključna briga.

> **Note**
>
> Za vašu profesionalnu praksu: SWE agenti neće zamijeniti programere, ali će fundamentalno promijeniti što znači biti programer. Sposobnost suradnje s autonomnim agentima, razumijevanja njihovih mogućnosti i ograničenja te učinkovitog vođenja njihovog rada postat će jednako važna vještina kao što je danas poznavanje programskog jezika ili razvojnog okvira.

6 Kod za generalističke agente

U prethodnom poglavlju govorili smo o agentima specijaliziranim za softversko inženjerstvo, sustavima koji pišu, testiraju i popravljaju kod unutar jasno definiranih razvojnih procesa. Sada podižemo pogled na širu sliku. Zamislite razliku između kirurga koji operira u operacijskoj sali i planinara koji se snalazi u divljini bez mape, bez staze, bez jasnog cilja osim preživjeti i napredovati. Generalistički agenti su ti planinari. Oni koriste kod ne samo za pisanje softvera, nego kao univerzalni medij koji im omogućuje razmišljanje o problemima i izvršavanje akcija kroz mnogo različitih zadataka i okruženja, umjesto da budu ograničeni na jednu specijaliziranu funkciju.

Integracija koda postala je ključna paradigma u razvoju generalističkih agenata, omogućujući kombinaciju kognitivnog rezoniranja s izvršivim akcijama kroz raznolike sredine. Ovaj odjeljak sintetizira nedavni napredak u korištenju koda izvan njegove konvencionalne uloge, naglašavajući tri dimenzije u arhitekturi agenata. Prva su protokoli interakcije, gdje kod uspostavlja strukturirane komunikacijske okvire uključujući obrasce korištenja alata, protokol konteksta modela (engl. *Model Context Protocol* ili MCP) i sheme koordinacije više agenata (engl. *Agent-to-Agent* ili A2A). Druga su agentske sposobnosti, gdje pristupi vođeni kodom omogućuju agentima generiranje i izvršavanje koda za složenu logiku, manipulaciju podatcima i zadatke softverskog

inženjerstva. Treća su sučelja okruženja, gdje kod podupire i simulirana okruženja i sučelja prema stvarnom svijetu, nudeći skalabilne i provjerljive platforme za treniranje, evaluaciju i postavljanje agenata.

6.1 Kod kao protokoli interakcije

6.1.1 Korištenje alata

Zamislite majstora u radionici. Oko njega su stotine alata, od čekića i pile do sofisticiranih elektroničkih instrumenata. Ono što ga čini majstorom nije samo znanje o svakom alatu, nego sposobnost da u pravom trenutku odabere pravi alat, pravilno ga uporabi i interpretira rezultat. Upravo tu analogiju možemo primijeniti na LLM agente koji koriste alate. U radnim procesima agenata baziranih na LLM-ovima, svaki alat je u osnovi utjelovljen u kodu koji služi kao formalni supstrat definirajući i sintaksno sučelje i semantičku logiku funkcionalnosti alata. Pozivanje funkcija (engl. *function calling*) djeluje kao središnji mehanizam, pružajući LLM-ovima strukturirano sučelje za interakciju s izvršivim kodom ili vanjskim uslugama.

Metode pozivanja funkcija mogu se široko kategorizirati u pristupe bez ugađanja i pristupe s ugađanjem. Pristupi bez ugađanja koriste sposobnosti učenja iz konteksta kod LLM-ova, tipično koristeći demonstracije s malo uzoraka ili usavršavanja bazirana na pravilima. Reverse Chain primjerice usvaja strategiju obrnutog rezoniranja koja najprije identificira odgovarajući ciljni alat za dani zadatak, a zatim popunjava relevantne parametre. EasyTool pojačava razumijevanje funkcija alata i zahtjeve parametara poticanjem ChatGPT-a da prepiše dugačke opise alata u sažete smjernice orijentirane na funkcije. Pristupi s ugađanjem poboljšavaju pozivanje funkcija kroz fino ugađanje parametara s namjenskim skupovima podataka za učenje alata. GPT4Tools integrira sposobnosti korištenja alata u LLM-ove otvorenog koda putem LoRA finog ugađanja, koristeći skupove podataka o upotrebi alata automatski generirane od strane ChatGPT-a. ToolkenGPT uvodi specijalizirane tokene, nazvane toolkeni, koji djeluju kao okidači za pozivanje alata.

Pozivanje funkcija služi kao temeljni mehanizam u raznim paradigmama agenata obogaćenih alatima Yao i ostali (2023). Okvir ReAct implementira pozive funkcija tijekom faze akcije, gdje model eksplicitno ispisuje naziv alata i parametre za pokretanje poziva, nakon čega slijedi faza promatranja u kojoj se odgovori alata integriraju za vođenje naknadnog rezoniranja. ReWOO nasuprot tome ima dvostupanjsku strukturu s fazom planiranja koja sastavlja strukturiranu listu potrebnih poziva funkcija i fazom izvršavanja u kojoj agenti radnici pozivaju funkcije, izvršavaju zadatke i vraćaju rezultate. Za rukovanje nesigurnošću i neuspjelim pozivima, DERA uvodi ciklički protokol interakcije. Kada se suoči s dvosmislenim uputama, model suspendira izvršavanje, ulazi u pojašnjavajući dijalog s korisnikom i nastavlja pozive funkcija nakon što stekne dovoljno informacija.

6.1.2 Protokol konteksta modela (MCP)

Protokol konteksta modela je standardizirani komunikacijski okvir dizajniran za koordinaciju interakcija između modela i vanjskih alata. Za razliku od autonomnog korištenja alata koje u potpunosti ovisi o internom rezoniranju, MCP uvodi strukturirane formate poruka, eksplicitnu semantiku pozivanja i dobro definirane mehanizme za upravljanje stanjima konteksta. Zamislite to kao standardizirani jezik kojim govore svi alati i svi modeli, poput engleskog jezika u međunarodnoj diplomaciji. Bez zajedničkog protokola, svaki par model i alat morao bi imati vlastiti prilagođeni način komunikacije, što bi bilo neodrživb pri skaliranju. Operativni ciklus MCP-a sastoji se od četiri faze: zahtjev, odgovor, ažuriranje stanja i ponovni poziv. Zajedno, ove faze tvore proces zatvorene petlje koji povezuje izvršavanje alata s kontinuiranim upravljanjem kontekstom, čime poboljšavaju pouzdanost, interpretabilnost i skalabilnost dovršavanja zadataka kroz više krugova. MCP je otvoren standard koji omogućuje kompozibilnost, dopuštajući graditeljem agenata da integrira više različitih alata i usluga kroz jedinstveni protokol.

6.1.3 Koordinacija više agenata

Agent-to-Agent ili A2A je suradnički obrazac u kojem agenti komuniciraju izravno jedni s drugima za ostvarivanje složenih zadataka. Umjesto da se oslanjaju na autonomno rezoniranje jednog agenta, A2A dekompozira zadatke među specijaliziranim agentima koji koordiniraju pozivanje alata i obradu informacija kroz prosljeđivanje poruka i dijeljeni kontekst. Zamislite tim specijalista u bolnici. Svaki liječnik ima svoju ekspertizu, ali zajedno donose odluke o liječenju pacijenta kroz konzilij, dijeleći nalaze i raspravljajući o opcijama. Upravo tako funkcionira A2A, agenti u ovoj paradigmi mogu pozivati vanjske alate i interagirati s kolegama agentima za popunjavanje praznina u znanju ili kompenziranje ograničenja, čime poboljšavaju i učinkovitost i točnost rješavanja problema.

6.2 Kod kao agentske sposobnosti

AI agenti su stekli znatan istraživački interes, popraćen rastućim naporima za karakterizaciju njihovih temeljnih sposobnosti. ReAct paradigma strukturira ponašanje agenta u tri faze: misao (rezoniranje, planiranje i donošenje odluka), akcija (izvršavanje odluka) i promatranje (primanje povratnih informacija iz okruženja za naknadne odluke). Ovaj odjeljak analizira ulogu LLM-ova baziranih na kodu unutar arhitekture agenata iz tri perspektive: razmišljanje, djelovanje i pamćenje.

6.2.1 Razmišljanje u kodu

Kada se suočavaju sa složenim zadacima rezoniranja, LLM-ovi značajno profitiraju od tehnika generiranja koda koje poboljšavaju i preciznost i učinkovitost. Zamislite razliku između studenta koji pokušava izračunati složen integral u glavi i studenta koji napiše program koji ga izračunava.

Drugi student ne mora držati sve međukorake u radnoj memoriji, jer ih je eksternalizirao u kod koji se može izvršiti i provjeriti. Koncept rezoniranja pomoću koda inicijalno je nastao u domeni matematičkog rješavanja problema. Programski potpomognuti jezični modeli (engl. *Program-Aided Language Models* ili PAL) koriste učenje iz malo uzoraka i poticanje lanca misli za generiranje Python koda koji izvršava međukorake rezoniranja uključujući matematičke operacije, simboličku manipulaciju i algoritmičku logiku. Ovaj pristup omogućuje točnije ishode kroz verifikaciju baziranu na izvršavanju.

Izvan matematičkog rješavanja problema, lanac koda (engl. *Chain of Code* ili CoC) okvir rezoniranja proširuje kodovno bazirano rezoniranje na numeričke i opće semantičke zadatke. LLM-ovi generiraju i izvršavaju isječke koda pod vodstvom upita, olakšavajući strukturirano rješavanje problema. Za numeričke zadatke, CoC omogućuje precizno računanje kroz izvršivi kod. Za semantičko rezoniranje koristi pseudokod sastavljen od struktura podataka, uvjeta i petlji za modeliranje apstraktnih procesa rezoniranja. Osim podržavanja rezoniranja za jedan zadatak, tehnike generiranja koda igraju ključnu ulogu u planiranju za više zadataka. CodePlan formulira planiranje zadataka eksplicitno u kodu, čime smanjuje dvosmislenost iz uputa na prirodnom jeziku i poboljšava performanse u složenim zadacima višekoraknog rezoniranja. Za višeagentske scenarije, namjenski planer generira planove pseudokoda u stilu Pythona koji obuhvaćaju inicijalizaciju varijabli, uvjetnu logiku i iterativne strukture, transformirajući ih u izvršive CodeAct instrukcije.

6.2.2 Djelovanje u kodu

Kada se koristi unutar izvršnog modula agenta, kod se mora povezati s alatima na razini sustava putem standardiziranih agentskih protokola Schick i ostali (2024). GPT-4 Code Interpreter omogućuje modelima izvršavanje koda za napredne zadatke rezoniranja. Open Interpreter predstavlja okvir otvorenog koda koji omogućuje jezičnim modelima lokalno izvršavanje koda kroz više jezika, uključujući Python, JavaScript i Shell. CodeAct uvodi koncept koji omogućuje interaktivne operacije generiranjem izvršivog Python koda, eliminirajući potrebu za prilagođenim omotačima alata izravnom integracijom izvršavanja koda s Python interpretatorom. Ovaj pristup nudi bogatiju podršku za tok kontrole i upravljanje podatcima, omogućujući pohranjivanje međurezultata kao varijabli i njihovo ponovno korištenje kroz korake.

Zanimljiv je nalaz o tome kako LLM-ovi s različitim sposobnostima rezoniranja odlučuju hoće li pozvati Code Interpreter. Eksperimenti na zadacima matematičkog rezoniranja otkrivaju kontraintuitivan trend. Manje sposobni modeli poput GPT-3.5 skloniji su korištenju Code Interpretera i posljedično postižu višu točnost. Nasuprot tome, sposobniji modeli poput GPT-4o teže se osloniti na vlastite interne sposobnosti rezoniranja i preferiraju tekstualno rezoniranje, često pokazujući pretjerano samopouzdanje koje vodi do povećanih grešaka na umjereno teškim problemima, fenomen nazvan efekt inverznog skaliranja. Za praktične primjene, nekoliko studija ispituje konkretne slučajeve uporabe. U zdravstvenom sektoru, EHRAgent omogućuje medicinskom osoblju izravno generiranje izvršivog koda putem LLM-ova za višetablično rezoniranje nad elektroničkim zdravstvenim zapisima. U domeni utjelovljene inteligencije, LLM generirani

kod koristi se za upravljanje robotskim sustavima, omogućujući agentima pretvaranje uputa visoke razine u precizne izvršive akcije.

6.2.3 Pamćenje kodom

Kao odgovor na ograničenja duljine konteksta kod LLM-ova, istraživači su istražili razne strategije pohrane, među kojima je pohrana bazirana na kodu pokazala zapaženu učinkovitost G. Wang i ostali (2023). Zamislite programera koji svaki put kada nauči nešto novo zapisuje rješenje u vlastitu biblioteku funkcija. Sljedeći put kada naiđe na sličan problem, ne mora razmišljati od nule, samo pozove gotovu funkciju iz svoje biblioteke. Upravo tako funkcionira pamćenje kodom. Voyager primjerice implementira okvir u kojem agenti autonomno stječu vještine generiranjem i iterativnim usavršavanjem koda kroz interakcije s okruženjem u Minecraftu. Validirane vještine pohranjuju se kao izvršivi kod u namjenskoj biblioteci i kasnije se izravno dohvaćaju za ponovnu uporabu, čime se eliminira potreba za ponovljenim zaključivanjem modela. MemGPT Peng i ostali (2023) predlaže okvir upravljanja kontekstom inspiriran virtualnom memorijom koji proširuje efektivnu memoriju LLM-ova dizajniranjem specijaliziranih poziva funkcija za čitanje i pisanje radi dinamičkog upravljanja i internim i vanjskim kontekstom.

6.3 Kod kao sučelja okruženja

6.3.1 Kod kao simulacijski poligon

S napretkom sposobnosti rezoniranja LLM-ova i rastućom raznolikošću zadataka rezoniranja, evaluacija i kultiviranje vještina dugoročnog planiranja postala je važan izazov. Tradicionalna jednokružna mjerila rezoniranja nedovoljna su za procjenu ili treniranje takvih sposobnosti. Da bi se riješilo ovo ograničenje, predložen je koncept koda kao simulacijskog poligona (engl. *Code as Simulation Gym* ili CodeGYM), izgrađen na Gymnasium platformi za pojačano učenje. CodeGYM konstruira raznovrsna složena okruženja zadataka koja omogućuju modelima kontinuiranu interakciju, održavanje i ažuriranje stanja te primanje povratnih informacija ili nagrada.

Dinamički poligon za zagonetke. Ova obitelj sustava za generiranje zagonetki baziranih na pravilima dizajnirana je za automatsku konstrukciju skupova podataka za rezoniranje velikog razmjera i provjerljive ispravnosti. Rani sustavi poput LogicGamea i KORBencha prezentiraju raznolike logičke zagonetke koje zahtijevaju od modela interpretaciju početnih stanja i primjenu specificiranih pravila. Napredniji sustavi koriste strategije transformacije i mehanizme ograničenja za proizvodnju složenih kombinatoričkih zadataka rezoniranja.

Sintetički prostorni poligon. Ovo je okvir za sintetičko generiranje podataka dizajniran za stvaranje prilagođenih skupova podataka za treniranje i evaluaciju LLM-ova na zadacima prostornog rezoniranja, čime se eliminiraju visoki troškovi povezani s prikupljanjem slika iz stvarnog svijeta. Rani primjeri uključuju mjerila za robotsko učenje, scenarije manipulacije iz

stvarnog svijeta i višezadaćna okruženja. S napretkom LLM-ova pojavile su se specijalizirane platforme za podršku prostornom rezoniranju.

Igračka arena. GameArena je klasa CodeGYM platformi dizajniranih za evaluaciju i treniranje sposobnosti dugoročnog planiranja i strateškog rezoniranja velikih jezičnih modela kroz interaktivna okruženja bazirana na igrama. Rani sustavi izgrađeni su na postojećim komercijalnim igrama uključujući StarCraft II, Minecraft i Black Myth Wukong. Naknadni rad proširio je paradigmu na igre socijalne dedukcije poput Avalona i Vukodlaka te narativne postavke igranja uloga, naglašavajući sposobnosti modela u strateškoj interakciji i socijalnom zaključivanju.

6.3.2 Agenti za korištenje računala

Kako LLM-ovi i vizualno-jezični modeli stječu jače sposobnosti rezoniranja, agenti sve više mogu autonomno djelovati u digitalnim okruženjima. Ovaj napredak doveo je do pojave agenata za korištenje računala, tipično kategoriziranih u tri vrste.

Agenti grafičkog sučelja (GUI agenti). GUI agenti interagiraju sa svojim okruženjima analizom sadržaja zaslona i generiranjem strukturiranih naredbi akcija u JSON formatu ili izvršivog koda za automatizaciju zadataka. Modul percepcije igra važnu ulogu obradom trenutnog stanja zaslona i històrijskog konteksta za omogućavanje korak po korak rezoniranja. Strategije percepcije uključuju tekstualne pristupe koji se oslanjaju na HTML strukture ili stabla pristupačnosti, slikovne pristupe koji se fokusiraju na interpretaciju vizualnog sadržaja izravno i multimodalne pristupe koji kombiniraju prednosti obaju modaliteta. Modul interakcije određuje prostor akcija agenta, prihvaća valjane naredbe i izvršava ih. Dvije glavne strategije su simulacija ljudskih akcija koja oponaša korisničke operacije uključujući klikove mišem i unos tipkovnice te interakcija bazirana na alatima koja izravno poziva programska sučelja ili izvršava skripte za postizanje veće učinkovitosti.

Agenti terminala. Agenti terminala interagiraju s okruženjima putem sučelja terminala ili ljuske, gdje prevode korisničke upute u kod naredbenog retka i izvršavaju ga. OB-1 gradi ansambl agenata opremljen dijeljenom memorijom, mehanizmima povratnih informacija i poticaja. Koristi trajne memorijske blokove za pohranu zadataka, bilješki i stavki za obaviti kroz dulja razdoblja, podržavajući uredive povijesti za ažuriranje prethodnog rezoniranja. Warp održava uredivu listu zadataka koja se dinamički ažurira s napretkom zadataka ili kada nastanu odstupanja.

Agenti za više okruženja. Agenti za više okruženja integriraju se s raznim sustavima putem mehanizma kodnih akcija, omogućujući interakciju kroz preglednike, terminale, izvršitelje koda i sustave datoteka. Manus koristi okvir preglednika i programska sučelja za dohvat podataka s interneta, izvršavajući kod u izoliranom virtualnom stroju za generiranje odgovora baziranih na HTML-u. OpenAI Deep Research pretražuje i interpretira sadržaj weba velikog razmjera, uključujući tekst, slike i PDF dokumente, za generiranje sveobuhvatnih izvješća. OpenManus kombinira planiranje vođeno LLM-om s izvršavanjem alata u višeagentskom sustavu, dok

OWL gradi okvir višestrukih agenata baziran na ulogama koji integrira specijalizirane agente i podržava više pretraživača i formata datoteka.

Za vašu profesionalnu praksu, ovo poglavlje otkriva jednu fundamentalnu istinu. Kod prestaje biti samo proizvod i postaje medij mišljenja, djelovanja i pamćenja za umjetnu inteligenciju. Kada agent razmišlja u kodu, djeluje u kodu i pamti u kodu, granice između programiranja i inteligencije postaju nejasne. Razumijevanje ove promjene važno je jer će odrediti kakve će alate koristiti sljedeća generacija programera i kakve će sposobnosti od njih zahtijevati. Tko razumije kako agenti koriste kod kao univerzalni medij, razumjet će i zašto budućnost softverskog inženjerstva izgleda fundamentalno drugačije od sadašnjosti.

> **Note**
>
> Za vašu profesionalnu praksu, ovo poglavlje otkriva jednu fundamentalnu istinu. Kod prestaje biti samo proizvod i postaje medij mišljenja, djelovanja i pamćenja za umjetnu inteligenciju. Kada agent razmišlja u kodu, djeluje u kodu i pamti u kodu, granice između programiranja i inteligencije postaju nejasne. Razumijevanje ove promjene važno je jer će odrediti kakve će alate koristiti sljedeća generacija programera i kakve će sposobnosti od njih zahtijevati.

7 Sigurnost kodnih LLM-ova

Zamislite da angažirate građevinskog inženjera koji projektira prekrasne mostove. Svaki most izgleda odličan, svaki detalj je estetski dotjeran. Ali postoji problem. Otprilike svaki drugi most koji dizajnira ima skrivenu strukturalnu slabost koju nitko ne primjećuje dok most ne počne nositi promet. To je, u grubo prevedenom obliku, situacija s velikim jezičnim modelima za kod. Oni transformiraju razvoj softvera. No kako njihove sposobnosti kodiranja napreduju, sigurnost generiranog koda postaje sve veća zabrinutost. Empirijske studije sustavno otkrivaju da otvoreni modeli poput Qwen3 i DeepSeek-R1 često isporučuju nesigurna rješenja. Štoviše, napredne tehnike neprijateljskog poticanja mogu navesti modele da proizvode funkcionalno ispravan ali suptilno ranjiv kod.

Sigurnosni krajolik kodnih LLM-ova predstavlja jedinstvene izazove različite od tradicionalnih softverskih ranjivosti, jer ovi modeli mogu propagirati i pojačati nesigurne obrasce kodiranja naučene iz svojih podataka za treniranje. Ovaj odjeljak pruža sveobuhvatan pregled izazova sigurnosti kodnih LLM-ova i strategija ublažavanja, organizirajući raspravu oko četiri kritična aspekta: sigurnost pretreniranja, sigurnost posttreniranja, evaluacija crvenog tima (engl. *red-teaming*), i sigurnost kodnih agentskih sustava.

7.1 Sigurnost pretreniranja kodnih LLM-ova

Faza pretreniranja uspostavlja temeljne sigurnosne karakteristike kodnih LLM-ova, čineći je kritičnom fazom za ugradnju sigurnosnih razmatranja u ponašanje modela. U korijenu, temelj nesigurnosti modela leži u podatcima za pretreniranje. Kodni LLM-ovi trenirani su na javnim repozitorijima gdje su nesigurni obrasci kodiranja uobičajeni. Zamislite studenta koji uči kuhati gledajući tisuće video zapisa na internetu. Većina tih videa pokazuje kuhare koji ne peru ruke, ne paze na temperature i koriste iste daske za meso i povrće. Student će naučiti pripremiti ukusna jela, ali s užasnim higijenskim navikama. Upravo to se događa modelima koji uče iz neprovjerenih repozitorija.

7.1.1 Porijeklo podataka, sigurnost i usklađenost s licencama

Sigurnost i pouzdanost kodnih LLM-ova fundamentalno su ukorijenjene u sigurnosti i legitimnosti njihovih korpusa za pretreniranje. Primarna zabrinutost je ublažavanje pravnih i etičkih rizika, posebno nenamjernog uključivanja koda pod restriktivnim licencama ili licencama s obvezom dijeljenja pod istim uvjetima (engl. *copyleft*). Pionirski napori poput BigCode projekta razvili su sustavne pristupe za kuriranje podataka. The Stack pruža 6.4 terabajta permisivno licenciranog koda nakon opsežne deduplikacije. StarCoder je dodatno usavršio ovaj pristup implementacijom sveobuhvatne strategije upravljanja podatcima.

7.1.2 Revizija i čišćenje podataka za treniranje

Nakon osiguravanja sigurnosti izvora podataka, sigurnost korpusa korištenih u pretreniranju jednako je ključna. Heuristički filtri koriste pravila obrazaca za otkrivanje poznatih iskorištivih ranjivosti. Drugi pristup uključuje postavljanje klasifikatora visokog kapaciteta koji funkcioniraju kao detektori sigurnosti. Sintetički primjeri odbijanja koriste se za pretvaranje opasnih upita u sigurna odbijanja.

7.1.3 Robusnost protiv neprijateljskih transformacija koda

Kritična dimenzija sigurnosti kodnih LLM-ova je robusnost protiv neprijateljskih napada koji specifično iskorištavaju jedinstvena svojstva izvornog koda. Za razliku od prirodnog jezika, formalna sintaksa i struktura programskih jezika stvaraju posebnu površinu napada, sintaksne transformacije koje čuvaju semantiku. Istraživanja su demonstrirala da se jednostavni zlonamjerni upiti koje sigurnosni filtri odbijaju mogu uspješno prikriti obfuskacijama specifičnim za kod. Primarna obrambena paradigma je neprijateljsko treniranje kroz proširenje podataka.

7.2 Sigurnost posttreniranja kodnih LLM-ova

Predominantni cilj pretreniranja kodnih LLM-ova ne usklađuje se dobro s generiranjem sigurnog koda. Ova paradigma potiče modele da uče i repliciraju uobičajene obrasce kodiranja iz golemih, nekuriranih skupova podataka, bez obzira na njihove sigurnosne implikacije. Zamislite liječnika koji zna sve o sterilizaciji instrumenata iz udžbenika ali u praksi nikada ne pere ruke jer je tako naučio gledajući starije kolege. Evaluacije koda generiranog LLM-ovima dosljedno otkrivaju visoke stope ranjivosti, pri čemu neke studije izvještavaju da otprilike 40% generiranih programa sadrži sigurnosne nedostatke.

7.2.1 Podatci kao temelj za sigurnost

Temelj svakog robustnog, sigurnosno usklađenog kodnog LLM-a su podatci iz kojih uči. Prevladavajuće metodologije za konstrukciju sigurnosno usmjerenih skupova podataka mogu se kategorizirati u tri paradigme. Prva je rudarenje parova popravaka ranjivosti iz softverskih repozitorija stvarnog svijeta povezivanjem predaja s bazom poznatih ranjivosti (CVE), čime su stvoreni skupovi podataka poput CVEfixes i BigVul. Druga je sintetičko generiranje podataka koje koristi vještog učiteljskog LLM-a za stvaranje golemih količina visokokvalitetnih, ciljanih instanci. Treća je destilacija podataka preferencija, konstruiranje trojki upita, odabranog sigurnog odgovora i odbačenog nesigurnog odgovora.

7.2.2 Sigurnosno nadzirano fino ugađanje

SFT baziran na sadržaju, gdje se model fino ugađa na parovima ranjivog i ispravljenog koda, učinkovit je. SFT baziran na uputama, gdje se model trenira na uputama koje eksplicitno opisuju sigurnosni zadatak, poboljšava sposobnost modela za prijenos naučenog znanja u nove kontekste. Najdinamičniji pristup je SFT baziran na povratnim informacijama koji integrira vanjske alate u petlju treniranja, koristeći alate za statičko testiranje sigurnosti aplikacija, jedinične testove ili metode formalne verifikacije.

7.2.3 Sigurnosno usklađivanje kodiranja putem pojačanog učenja

Pojačano učenje nudi moćnu paradigmu za dinamičko usklađivanje kodnih LLM-ova sa složenim sigurnosnim ciljevima. RLHF je predominantni pristup u tri faze. Safe RLHF dodatno usavršava ovo razdvajanjem procesa optimizacije, trenirajući zasebne modele za korisnost (nagrada) i neškodljivost (trošak). Konstruiranje robustnih signala nagrade uključuje provjerljive povratne informacije iz alata (kompiliranje, prolaz testova, alati za statičku analizu), povratne informacije generirane AI-jem i hibridne arhitekture nagrada.

7.3 Tehnike crvenog tima za kodne LLM-ove

Procjena sigurnosti kodnih LLM-ova postaje sve izazovnija kako se pojavljuju nove neprijateljske tehnike. Zamislite to kao igricu mačke i miša u kojoj napadači neprestano pronalaze nove rupe u ogradi, a branitelji ih pokušavaju zatvoriti. No svaka nova ograda ima nove šavove, i upravo te šavove napadači najprije ispituju.

7.3.1 Manipulacija na razini upita

Ova kategorija obuhvaća napade koji konstruiraju neprijateljske ulaze dizajnirane za podrivanje sigurnosnog usklađivanja modela. Heuristički jailbreaking koristi scenarije igranja uloge u kojima se model upućuje da usvoji personu lišenu uobičajenih etičkih ograničenja. Napadi neprijateljske optimizacije formuliraju generiranje neprijateljskih upita kao problem optimizacije. GCG algoritam koristi informacije o gradijentima za iterativno usavršavanje sekvence znakova. Konverzacijski i višekružni napadi distribuiraju napad kroz više koraka dijaloga.

7.3.2 Semantička i kontekstualna manipulacija

Ova sofisticiranija klasa napada iskorištava nesklad između sintaksnog filtriranja i semantičkog razumijevanja. Iskorištavanje granice povjerenja nastaje kada protivnik pruži bezopasnu naredbu visoke razine dok ugrađuje stvarni zlonamjerni sadržaj unutar izvora podataka. Neizravno ubacivanje upita je napad posebno kritičan za agentske sustave koji interagiraju s vanjskim podatcima. Obfuskacija i međujezični napadi ciljaju zaobilaženje sigurnosnih modela.

7.3.3 Manipulacija agentskog tijeka rada

Pojava LLM agenata koji mogu izvršavati alate i interagirati sa živim okruženjima uvodi novu površinu napada fokusiranu na subverziju akcija. Zlouporaba alata i ubacivanje zlonamjernih argumenata nastaje kada protivnik oblikuje upite koji navode agenta da proslijedi zlonamjerne, nesanitarizirane argumente. Bijeg iz izoliranog okruženja (engl. *sandbox escape*) usmjerava agenta da generira i izvrši kod koji ispituje i iskorištava ranjivosti. Automatizirano iskorištavanje ranjivosti predstavlja vrhunac agentskih sposobnosti.

7.4 Strategije ublažavanja za rizike kodiranja

Za rješavanje višestrukih rizika koje predstavljaju autonomni i poluautonomni agenti koji generiraju kod, ključna je robusna, višeslojna sigurnosna paradigma. Okvir obrane u dubinu (engl. *Defense-in-Depth*) pomiče fokus od pukog validiranja ispravnosti koda prema holističkom upravljanju ponašanjem agenata kroz tri međuovisna sloja.

7.4.1 Temelji u sigurnim okruženjima za izvršavanje

Prva linija obrane je uspostavljanje stroge izolacijske granice koja ograničava potencijalni utjecaj agenta. Spektar izolacijskih tehnologija uključuje kontejnere na razini operativnog sustava poput Dockera, izolirane okruženja na razini procesa poput nsjaila, i mikro virtualne strojeve poput Firecrackera koji nude izolaciju blizu hardverske razine. Kritično ograničenje ovih pristupa je njihova statična priroda jer unaprijed definirana politika često stvara jaz sposobnosti.

7.4.2 Proaktivna obrana i validacija prije izvršavanja

Drugi sloj obrane fokusira se na preventivnu provjeru artefakata i planova generiranih od strane agenata prije njihovog izvršavanja. Modernizirane analize koda integriraju alate za statičko i dinamičko testiranje sigurnosti aplikacija u ciklus povratnih informacija. Višeagentski pregled i suradnja, inspirirani ljudskim praksama poput pregleda koda, koriste razne suradničke obrasce. Formalne metode i verifikacija namjere predstavljaju najrigorozniji oblik validacije prije izvršavanja.

7.4.3 Nadzor u izvršavanju i utemeljenje namjere

Završni obrambeni sloj rješava rizike koji se dinamički manifestiraju tijekom izvršavanja. Srž problema je da sintaksno ispravne i lokalno optimalne operacije mogu kaskadirati u globalno nesigurne ili nepovratne promjene stanja. Okviri zaštitnih ograda poput AgentSentinela pružaju cjelovito sigurnosno praćenje u stvarnom vremenu. Čuvarski agent (engl. *guardian agent*) je sekundarni LLM fokusiran na sigurnost koji promatra akcije primarnog agenta. Provjediva primjena politika putem okvira osigurava da ponašanje agenta ostaje unutar formalno definiranih granica.

> **Note**
>
> Za vašu profesionalnu praksu: Kodni LLM-ovi su nesigurni po pretpostavci. Njihovo treniranje na javnim repozitorijima, gdje je manjkav kod sveprisutan, predisponira ih da slijede put najmanjeg otpora. Sigurnost postaje vanjski cilj koji se natječe s primarnim ciljem modela. Kada koristite alate za generiranje koda, ne pitajte samo radi li kod. Pitajte je li siguran. Ta dva pitanja nisu isto.

8 Recepti za treniranje

Zamislite da želite ispeći savršeni kruh. Ne bilo kakav kruh, nego onakav kakav peče majstor pekar koji je proveo trideset godina usavršavajući svoj zanat. Trebat će vam recept, ali ne

samo popis sastojaka, nego i precizne temperature, vremena miješanja, opis teksture tijesta u svakoj fazi. Upravo to ovaj odjeljak nastoji pružiti za treniranje kodnih LLM-ova. Treniranje vrhunskog kodnog LLM-a sofisticiran je i višefazni cjevovod u kojem svaka faza služi zasebnim svrhama.

Proces tipično počinje pretreniranjem, gdje LLM uči temeljne statističke obrasce programskih jezika iz golemih baza koda. Nakon pretreniranja, model prolazi nadzirano fino ugađanje (SFT) za prilagodbu općih sposobnosti. Na kraju, model se može dodatno usavršiti korištenjem pojačanog učenja (RL).

8.1 Distribuirani okviri za treniranje

Zamislite da gradite piramidu. Jedan radnik može nositi jedan kamen odjednom i za završetak mu trebaju stoljeća. Sto radnika može nositi sto kamenova odjednom, ali samo ako su savršeno koordinirani. Upravo taj izazov koordinacije stoji iza distribuiranih okvira za treniranje.

Megatron-LM Shoeybi i ostali (2020) uvodi učinkovit pristup paralelizmu modela unutar slojeva koji particionira pojedinačne transformerske slojeve kroz više uređaja. Temeljna inovacija leži u strategiji tenzorskog paralelizma. Ovaj dizajn zahtijeva samo minimalne operacije all-reduce komunikacije. Okvir implementira tri komplementarne tehnike paralelizacije, tenzorski paralelizam, cjevovodni paralelizam s mikrobatch izvršavanjem, i sekvencijski paralelizam.

DeepSpeed Rajbhandari i ostali (2020) se temelji na optimizatoru bez redundancije (engl. *Zero Redundancy Optimizer* ili ZeRO) koji eliminira memorijske redundancije u podatkovnom paralelnom treniranju. ZeRO-1 particionira stanja optimizatora za 4 puta smanjenje memorije. ZeRO-2 proširuje particioniranje na gradijente postižući 8 puta smanjenje. ZeRO-3 particionira parametre modela omogućujući linearno skaliranje s brojem uređaja.

PyTorch FSDP implementira ZeRO-3 optimizaciju kao izvornu PyTorch komponentu. FSDP2 redizajnira implementaciju koristeći DTensor apstrakcije. **TorchTitan** pruža proizvodnu referentnu implementaciju 4D paralelizma za pretreniranje LLM-ova. **Colossal-AI** pruža raznolike strategije paralelizacije s naglaskom na višedimenzionalnom tenzorskom paralelizmu.

8.2 Smjernice za pretreniranje

Pretreniranje predstavlja temeljnu fazu razvoja kodnog LLM-a, gdje modeli stječu fundamentalno programersko znanje iz golemih repozitorija koda. Zakoni skaliranja izvedeni iz opsežnih višejezičnih eksperimenata neprocjenjiv su alat za pružanje smjernica temeljenih na podatcima.

Jezično specifični zakoni skaliranja. Programski jezici pokazuju fundamentalno različita ponašanja skaliranja. Interpretirani jezici pokazuju veće eksponente skaliranja od kompiliranih jezika. Python demonstrira najviše vrijednosti eksponenata, ukazujući na agresivne koriti od povećanog kapaciteta modela. Statički tipizirani kompilirani jezici poput Rusta i Goa pokazuju

zamjetno manje eksponente, jer njihove krute sintaksne strukture i eksplicitne anotacije tipova nose više informacija po tokenu. Nereducibilni gubitak otkriva intrinzičnu složenost jezika.

Učinci višejezičnog miješanja. Sustavni eksperimenti na dvojezičnim mješavinama otkrivaju da višejezično treniranje pruža značajne prednosti nad jednojezičnim polaznim modelima. Jezici koji dijele sličnu sintaksu i semantiku pokazuju snažnu pozitivnu sinergiju. Primjerice, mješavine Java/C# postižu preko 20% smanjenja gubitka. Međutim, Python predstavlja asimetričnu iznimku. Miješanje Pythona s većinom statički tipiziranih jezika proizvodi male negativne učinke.

Preporučene strategije pretreniranja. Alociranje budžeta tokena proporcionalno eksponentima skaliranja umjesto uniformno. Jezici s visokim eksponentima značajno profitiraju od povećanih podataka. Prioritiziranje sintaksno sličnih jezičnih parova u mješavinama. Usmjeravanje produljenog treniranja na jezike s visokim nereducibilnim gubitkom. Višejezično pretreniranje trebalo bi biti zadani pristup jer većina jezika pokazuje dosljednu pozitivnu sinergiju.

8.3 Smjernice za fino ugađanje

Uspostavivši temeljne okvire za pretreniranje velikog razmjera, fokus se pomiče na nadzirano fino ugađanje. Dok pretreniranje gradi opće sposobnosti modela, SFT je ključan za prilagodbu specifičnim, visoko vrijednim zadacima. Ova faza prilagodbe uvodi vlastiti skup izazova i kompromisa.

Smjernice za okvire treniranja. Učinkovito SFT kodnih LLM-ova zahtijeva okvire koji balansiraju učinkovitost treniranja i performanse modela. Za SFT velikog razmjera s više epoha, okviri koji kombiniraju particioniranu memoriju s tenzorskim paralelizmom unutar sloja postižu najbolju ravnotežu.

Smjernice za parametre treniranja. Globalna veličina serije dominantan je čimbenik osjetljivosti za nadzirani SFT koda. Točnost degradira kada globalna serija premašuje otprilike 256. Optimalne stope učenja ovise o modelu. Duljina treniranja interagira sa skalama. Raspored učenja i zagrijavanje sekundarni su za manje modele ali važniji za veće, gdje je konstantni raspored sa skromnim zagrijavanjem sigurniji.

Smjernice za izbor skupova podataka. Sveobuhvatna usporedba javno dostupnih SFT skupova podataka otkriva značajne razlike u performansama kroz modele. Za gusti model, najviše performanse ostvaruju skupovi koji kombiniraju strukturirane programerske zadatke s raznolikim izvorima koda.

8.4 Smjernice za pojačano učenje

Nadzirano fino ugađanje visoko je učinkovito za podučavanje modela da oponaša distribuciju danog skupa podataka. Međutim, za zadatke poput generiranja koda, ispravnost se ne odnosi samo na stilove nego na objektivne i provjerljive ishode poput prolaska jediničnih testova.

Validacija estimatora prednosti. Izbor estimatora prednosti fundamentalan je za stabilnost i učinkovitost uzorkovanja algoritama gradijenata politike. RLOO postiže najbolju Pass@5 izvedbu. Međutim, REINFORCE++ baseline konvergira otprilike 30% brže uz održavanje konkurentne izvedbe, pokazujući stabilniju dinamiku treniranja. REINFORCE++ baseline preporučuje se kao zadani estimator.

Skaliranje maksimalne duljine odgovora. Zadaci generiranja koda mogu zahtijevati znatno različite duljine konteksta. Konfiguracija od 16K tokena postiže najvišu Pass@1 ocjenu. Zanimljivo, postavka od 2K tokena postiže najbolju Pass@5 izvedbu. Preporuka je 2K tokena za ciljeve teškog istraživanja, 16K tokena za Pass@1 i 4K tokena kao uravnotežena zadana vrijednost.

Skaliranje broja uzorkovanja po upitu. Broj odgovora uzorkovanih po upitu izravno utječe na širinu istraživanja i kvalitetu procjene prednosti. N=8 isporučuje najbolju Pass@5 izvedbu. Za Pass@1, rezultati pokazuju izvanrednu stabilnost. N=16 preporučuje se kao zadana konfiguracija zbog izvrsnog balansa između brzine konvergencije, računalne učinkovitosti i konkurentne izvedbe.

9 Veliki jezični modeli za kod u primjeni

Sve što smo opisali u prethodnih osam poglavlja, od arhitektura modela i zadataka pretreniranja, preko mjerila i evaluacije, usklađivanja, agenata, sigurnosti i recepata za treniranje, kulminira u jednom pitanju. Kako to izgleda u rukama stvarnih programera? Zamislite da ste proveli godine projektirajući motor automobila, testirajući ga u laboratoriju, mjereći svaki parametar do posljednje decimale. Sve to ima smisla samo ako motor na kraju završi pod haubom automobila koji stvarni ljudi voze po stvarnim cestama. Upravo o tome govori ovo poglavlje.

9.1 Razvojni pomoćnici integrirani u IDE

Razvojni pomoćnici integrirani u IDE predstavljaju najšire primijenjenu kategoriju aplikacija kodnih LLM-ova, bešavno ugrađujući AI sposobnosti unutar etabliranih razvojnih okruženja poput Visual Studio Codea, JetBrains IDE-ova i vlasničkih uređivača. Ovi alati ciljaju profesionalne programere koji rade na projektima, naglašavajući duboko kontekstualno razumijevanje, međudatotečno rezoniranje i integraciju s postojećim radnim procesima softverskog inženjerstva.

Alat	Ključne Karakteristike	Korisničke Baze
GitHub Copilot	GPT-4.1 po zadanoj postavci, Claude Opus 4 na Pro+, Edits, Workspace, agentski način	1.8M+ pretplatnika
Cursor	Vlastiti Tab model, Composer Mode, Agent Mode, dinamička prioritizacija konteksta	500M+ ARR za 2 godine
TRAE	Builder/SOLO Mode, CUE značajka, pametno preimenovanje, multimodalni unos	AI-native IDE
Tabnine	Fokus na privatnost, lokalni poslužitelji, permisivna licenciranja	Sigurnost-usmjereno
Windsurf	Kaskadna arhitektura, duboko razumijevanje baze koda, koordinacija više agenata	Produkcijsko okruženje
Cody (Sourcegraph)	Duboka integracija s pretraživanjem koda, superior dohvat u monorepo strukturama	Enterprise

GitHub Copilot pojavio se kao pionirski komercijalni AI pomoćnik za kodiranje integriran u IDE, prvotno lansiran u tehničkom pregledu 29. lipnja 2021. Izgrađen na OpenAI-jevom Codex modelu, Copilot je transformirao očekivanja programera od AI potpomognutog programiranja. Do 2025. godine, Copilot prema zadanim postavkama koristi GPT-4.1 kroz razgovor, agentski način i dovršavanje koda. Pro+ i Enterprise razine pružaju pristup graničnim modelima. Arhitektura obuhvaća sofisticirani agentski proces baziran na Node.js-u koji rukuje web zahtjevima, prikupljanjem konteksta i inženjeringom upita za LLM. Ključne inovacije uključuju Copilot Edits za višedatotečno refaktoriranje, Copilot Workspace za razvoj cjelovitih značajki i agentski način rada lansiran u svibnju 2025. Sustav obrađuje približno 150 milijuna prijedloga koda dnevno s zahtjevima latencije ispod jedne sekunde.

Cursor predstavlja strateško preispitivanje AI integracije u razvojna okruženja. Osnovan 2022. godine, lansiran u ožujku 2023. kao fork Visual Studio Codea, Cursor je prešao 500 milijuna dolara godišnjeg ponavljajućeg prihoda unutar samo dvije godine. Cursorova tehnička arhitektura centrirana je na vlastitim inovacijama u upravljanju kontekstom. Tab model pruža dovršavanja ultraniskih latencija ispod jedne sekunde implementirajući sofisticirani mehanizam sinkronizacije. Indeksiranje repozitorija koristi Merkle stabla za učinkovito praćenje promjena

datoteka. Sustav podržava kontekstne prozore do 200K tokena kroz dinamičku prioritizaciju konteksta. Composer Mode omogućuje uređivanja kroz čitav projekt na prirodnom jeziku. Agent Mode pruža autonomne sposobnosti kodiranja.

TRAE predstavlja novu generaciju AI-nativnih integriranih razvojnih okruženja. Proizvod manifestira viziju kroz dva primarna načina, Builder Mode i SOLO Mode. SOLO Mode funkcionira kao sveobuhvatni kontekstni inženjer. CUE značajka podržava automatsko dovršavanje, višelinijska uređivanja i pametno preimenovanje.

Tabnine se razlikuje fokusom na privatnost, sigurnost i usklađenost s intelektualnim vlasništvom, koristeći isključivo permisivno licencirani kod za treniranje modela i nudeći postavljanje na lokalne poslužitelje. Windsurf je nastao u studenom 2024. uvodeći kaskadnu arhitekturu za duboko razumijevanje baze koda i koordinaciju više agenata. CodeGeeX prioritizira višejezičnu podršku. Cody od Sourcegrapha integrira se duboko s infrastrukturom pretraživanja koda za superiorniji dohvat konteksta u monorepo strukturama. Bito AI naglašava privatnost s načinima izvanmrežnog rada.

9.2 Platforme za kodiranje u oblaku

Platforme za kodiranje u oblaku koriste skalabilnu infrastrukturu za pružanje usluga generiranja koda kroz programska sučelja, web sučelja i razvojna okruženja integrirana u oblak. Ovi sustavi ciljaju organizacije kojima je potrebno centralizirano postavljanje, dosljedne sigurnosne politike i duboka integracija s uslugama oblaka, posebno za infrastrukturu kao kod i razvoj aplikacija nativnih za oblak.

Amazon Q Developer predstavlja evoluciju Amazon CodeWhisperera. U travnju 2024. AWS je spojio CodeWhisperer u širu obitelj Amazon Q. Sustav se razlikuje dubokom specijalizacijom za AWS usluge, treniran na milijunima internih Amazon repozitorija koda, AWS dokumentaciji i implementacijama usluga. Arhitektura se integrira kroz čitav životni ciklus razvoja, od prijedloga koda u IDE-ovima, kroz sučelja za razgovor u AWS konzoli, do automatskog dovršavanja u naredbenom retku za AWS naredbe.

Google Cloud Code i Gemini Code Assist Googleove ponude za kodiranje u oblaku evoluirale su od Google Cloud Codea prema Gemini Code Assist, koristeći najnovije modele iz Gemini obitelji. Sustav se ističe u generiranju GKE konfiguracija, Cloud Run postavljanja i BigQuery SQL-a. Gemini CLI pruža razvoj nativni za terminal s brzim zaključivanjem. **Replit Ghostwriter** je pionir u AI potpomognutom razvoju u pregledniku. **Alibaba Tongyi Lingma** služi kinesko tržište sa specijaliziranom podrškom za kineske programerske kontekste.

9.3 Autonomni agenti bazirani na terminalu

Agenti bazirani na terminalu djeluju u okruženjima naredbenog retka, pružajući autonomne sposobnosti generiranja koda, modifikacije i upravljanja projektima. Ovi alati privlače programere

koji preferiraju radne procese vođene tipkovnicom, inženjere za automatizaciju i istraživače kojima su potrebni skriptabilni, ponovljivi eksperimenti.

Aider se uspostavlja kao vodeći agentski alat za kodiranje u terminalu otvorenog koda. Njegov sustav mapiranja repozitorija koristi tree-sitter za jezično agnostičko parsiranje apstraktnog sintaksnog stabla. Aider implementira više pozadina za uređivanje koda s automatskim odabirom formata i iterativni mehanizam povratnih informacija iz izvršavanja.

Claude Code, najavljen u prosincu 2024. od strane Anthropica, predstavlja razvojno okruženje nativno za terminal specifično dizajnirano za obitelj modela Claude. Sustav implementira Model Context Protocol (MCP), otvoreni standard koji omogućuje proširivo integriranje alata. Arhitektura naglašava komponabilnost gdje programeri mogu definirati prilagođene alate koje Claude može autonomno pozivati. Sposobnost planiranja agenta razbija složene zadatke u podciljeve s eksplicitnim generiranjem koraka.

Gemini CLI pruža pristup Googleovim Gemini modelima nativno za terminal s naglaskom na brzinu i učinkovitost. **Plandex** implementira distinktivan radni proces baziran na planovima s podrškom za grananje za istraživanje alternativnih implementacija.

9.4 Aplikacije za popravak i verifikaciju koda

Specijalizirane aplikacije za automatizirano popravljanje grešaka, zakrpavanje ranjivosti i formalnu verifikaciju rješavaju kritične potrebe pouzdanosti softvera. Ovi alati služe sigurnosnim timovima koji provode procjenu ranjivosti, QA inženjerima i istraživačima u formalnim metodama.

RepairAgent je pionir u autonomnom uklanjanju grešaka kroz progresiju stroja stanja vođenu hipotezama. Sustav implementira strukturirani pristup popravljanju grešaka u pet faza, lokalizacija greške, generiranje hipoteza, generiranje zakrpe, validacija i iterativno usavršavanje. **AutoSpec** rješava izazovan problem automatske sinteze specifikacija za formalnu verifikaciju. **AlphaRepair** kombinira analizu programa s LLM-ovima za superiornu lokalizaciju grešaka i generiranje zakrpi. **Toggle** izvodi lokalizaciju grešaka na razini tokena.

9.5 Pregled zahtjeva za spajanje i osiguranje kvalitete

Pomoćnici za pregled zahtjeva za spajanje (engl. *pull request review*) automatiziraju procese pregleda koda, pružajući povratne informacije o kvaliteti koda, potencijalnim greškama, sigurnosnim problemima i pridržavanju standarda kodiranja.

PR-Agent (Qodo-AI) pojavio se kao vodeći sustav za automatizirani pregled zahtjeva za spajanje otvorenog koda. Sustav se integrira kao korak u CI/CD cjevovodu, automatski se pokrećući pri stvaranju ili ažuriranju zahtjeva za spajanje. PR-Agent koristi više specijaliziranih upita za različite aspekte pregleda, arhitektonski pregled koji analizira strukturalne promjene, sigurnosni pregled, stilski pregled i logički pregled.

CodeRabbit pruža automatizirane preglede zahtjeva za spajanje s umetnutim prijedlozima, objašnjenjima koda i komentarima svjesnima konteksta. Sustav se ističe podrškom za dugačke razlike konteksta. **LLM Code Reviewer** pruža jednostavnu GitHub Action integraciju. **Graphite Reviewer** naglašava brzinu i prisjećanje konteksta kroz indeksiranje svjesno repozitorija. **Codedog** nudi višeplatformsku podršku s jakom UI integracijom za kineske razvojne timove.

Ovdje se zatvara krug čitavog teksta. Krenuli smo od temeljnih modela i njihovih arhitektura, prošli kroz zadatke i mjerila, usklađivanje, agente, sigurnost i recepte za treniranje, i završavamo tamo gdje sve to zapravo živi, u rukama programera koji koriste ove alate svaki dan.

> **Note**
>
> Za vašu profesionalnu praksu, poruka je jasna. Ovi alati nisu budućnost. Oni su sadašnjost. GitHub Copilot obrađuje 150 milijuna prijedloga koda dnevno. Cursor je premašio 500 milijuna dolara godišnjeg prihoda unutar dvije godine. Claude Code, Gemini CLI i Aider transformiraju terminalske radne procese. PR-Agent i CodeRabbit automatiziraju preglede koda. RepairAgent autonomno popravlja greške. Pitanje više nije hoće li ovi alati promijeniti način na koji radite. Pitanje je jeste li već počeli razumijevati kako rade, što mogu i ne mogu, i kako ih najučinkovitije koristiti. Odgovor na to pitanje određuje vašu profesionalnu relevantnost u godinama koje dolaze.

Reference

BUAA-SKLCCSE, Alibaba, ByteDance, M-A-P, BJTU, OPPO, HKUST (GZ), BUPT, TeleAI, Shanghai AI Lab, Manchester, StepFun, UoS, SCU, CASIA, NJU, Kuaishou, HIT, Huawei Cloud, Tencent, Monash/CSIRO, NTU, ZJU, BIT, Ubiquant, NUS, HNU, PKU, CSU. 2025. „From Code Foundation Models to Agents and Applications: A Comprehensive Survey and Practical Guide to Code Intelligence“. *arXiv preprint arXiv:2511.18538v5*. https://arxiv.org/abs/2511.18538.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, i ostali. 2021. „Evaluating large language models trained on code“. *arXiv preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374.

DeepSeek-AI. 2024. „DeepSeek-V3 Technical Report“. *arXiv preprint arXiv:2412.19437*. https://arxiv.org/abs/2412.19437.

———. 2025. „DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning“. *arXiv preprint arXiv:2501.12948*. https://arxiv.org/abs/2501.12948.

Feng, Zhangyin, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, i ostali. 2020. „CodeBERT: A pre-trained model for programming and natural languages“. *arXiv preprint arXiv:2002.08155*. https://arxiv.org/abs/2002.08155.

Gu, Albert, i Tri Dao. 2024. „Mamba: Linear-time sequence modeling with selective state spaces“. *arXiv preprint arXiv:2312.00752*. https://arxiv.org/abs/2312.00752.

Hong, Sirui, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, i ostali. 2024. „MetaGPT: Meta programming for a multi-agent collaborative framework“. *arXiv preprint arXiv:2308.00352*. https://arxiv.org/abs/2308.00352.

Hui, Binyuan, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, i ostali. 2024. „Qwen2.5-Coder Technical Report“. *arXiv preprint arXiv:2409.12186*. https://arxiv.org/abs/2409.12186.

Jimenez, Carlos E, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, i Karthik Narasimhan. 2024. „SWE-bench: Can language models resolve real-world GitHub issues¿‘ *arXiv preprint arXiv:2310.06770*. https://arxiv.org/abs/2310.06770.

Li, Raymond, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, i ostali. 2023. „StarCoder: may the source be with you¡‘ *arXiv preprint arXiv:2305.06161*. https://arxiv.org/abs/2305.06161.

Li, Yujia, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, i ostali. 2022. „Competition-level code generation with AlphaCode“. *Science* 378 (6624): 1092–97. https://doi.org/10.1126/science.abq1158.

Lozhkov, Anton, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, i ostali. 2024. „StarCoder 2 and The Stack v2: The next generation“. *arXiv preprint arXiv:2402.19173*. https://arxiv.org/abs/2402.19173.

Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, i ostali. 2022. „Training language models to follow instructions with human feedback“. *Advances in Neural Information Processing Systems* 35. https://arxiv.org/abs/2203.02155.

Peng, Charles, Yuxuan Xia, Yin Wu, i Sarah Zhu. 2023. „MemGPT: Towards LLMs as operating systems“. *arXiv preprint arXiv:2310.08560*. https://arxiv.org/abs/2310.08560.

Qian, Chen, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, i ostali. 2024. „ChatDev: Communicative agents for software development“. *arXiv preprint arXiv:2307.07924*. https://arxiv.org/abs/2307.07924.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, i Chelsea Finn. 2023. „Direct preference optimization: Your language model is secretly a reward model“. *Advances in Neural Information Processing Systems* 36. https://arxiv.org/abs/2305.18290.

Rajbhandari, Samyam, Jeff Rasley, Markus N Rabe, i Yuxiong He. 2020. „ZeRO: Memory optimizations toward training trillion parameter models“. *arXiv preprint arXiv:1910.02054*. https://arxiv.org/abs/1910.02054.

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, i Thomas Scialom. 2024. „Toolformer: Language

models can teach themselves to use tools“. *Advances in Neural Information Processing Systems* 36. https://arxiv.org/abs/2302.04761.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, i Oleg Klimov. 2017. „Proximal policy optimization algorithms“. *arXiv preprint arXiv:1707.06347*. https://arxiv.org/abs/1707.06347.

Shao, Zhihong, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, i Daya Guo. 2024. „DeepSeekMath: Pushing the limits of mathematical reasoning in open language models“. *arXiv preprint arXiv:2402.03300*. https://arxiv.org/abs/2402.03300.

Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, i Bryan Catanzaro. 2020. „Megatron-LM: Training multi-billion parameter language models using model parallelism“. *arXiv preprint arXiv:1909.08053*. https://arxiv.org/abs/1909.08053.

Touvron, Hugo, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, i ostali. 2023. „Llama 2: Open foundation and fine-tuned chat models“. *arXiv preprint arXiv:2307.09288*. https://arxiv.org/abs/2307.09288.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, i Illia Polosukhin. 2017. „Attention is all you need“. *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.

Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, i Anima Anandkumar. 2023. „Voyager: An open-ended embodied agent with large language models“. *arXiv preprint arXiv:2305.16291*. https://arxiv.org/abs/2305.16291.

Wang, Xingyao, Boxuan Hoang, Yufan Hu, Jialu Liu, Yuhang Liu, Fangzhou Liu, Kaiyu Deng, i ostali. 2024. „OpenHands: An open platform for AI software developers as generalist agents“. *arXiv preprint arXiv:2407.16741*. https://arxiv.org/abs/2407.16741.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, i Denny Zhou. 2022. „Chain-of-thought prompting elicits reasoning in large language models“. *Advances in Neural Information Processing Systems* 35. https://arxiv.org/abs/2201.11903.

Yang, John, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, i Ofir Press. 2024. „SWE-agent: Agent-computer interfaces enable automated software engineering“. *arXiv preprint arXiv:2405.15793*. https://arxiv.org/abs/2405.15793.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, i Yuan Cao. 2023. „ReAct: Synergizing reasoning and acting in language models“. *arXiv preprint arXiv:2210.03629*. https://arxiv.org/abs/2210.03629.